Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial

Implementasi Algoritma *Local Search*
untuk Optimasi Pemuatan Paket dalam Truk

Semester I Tahun Akademik 2026/2027



Kelompok G55 Tocatdz

Juan Oloando Simanungkalit	13524032
Narendra Dharma Wistara M.	13524044
Reinhard Alfonzo Hutabarat	13524056

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Daftar Isi

1 Deskripsi Persoalan **1**

1.1 Latar Belakang dan Tujuan 1
1.2 Definisi Permasalahan dan Data Masukan 1
1.3 Ketentuan dan Batasan Persoalan 1
1.4 Keluaran Program 2

2 Perancangan Solusi **2**

2.1 Representasi Paket, Truk, dan State 2
2.2 Pembentukan Initial State 2
2.3 Pembentukan Neighbor 3
2.4 Fungsi Objektif 3
2.5 Pemeriksaan Validitas State 3
2.5.1 Batas Ruang dan Tumpang Tindih 3
2.5.2 Support dan Paket Melayang 3
2.5.3 Larangan Fragile Menjadi Penopang 4
2.5.4 Kapasitas Berat 4
2.6 Rancangan Algoritma Local Search 4
2.6.1 Hill-Climbing 4
2.6.2 Simulated Annealing 5
2.6.3 Genetic Algorithm 5
2.7 Format Input dan Syarat Validitas 6

3 Implementasi Program **7**

3.1 Lingkungan dan Struktur Program 7
3.2 Initial State, Neighbor, dan Fungsi Objektif 8
3.3 Implementasi Hill-Climbing 10
3.3.1 Steepest-Ascent Hill-Climbing 10
3.3.2 Stochastic Hill-Climbing 11
3.3.3 Hill-Climbing with Sideways Move 11
3.3.4 Random-Restart Hill-Climbing 12
3.4 Implementasi Simulated Annealing 12
3.5 Implementasi Genetic Algorithm 13
3.5.1 Representasi Kromosom dan Individu 13
3.5.2 Decoding dan Evaluasi Fitness 14
3.5.3 Seleksi Induk 15
3.5.4 Crossover 15
3.5.5 Mutasi 16
3.5.6 Pembentukan Generasi dan Penghentian 16
3.5.7 Pencatatan Hasil Pencarian 16
3.6 Visualisasi dan Penyimpanan Hasil 17

4 Eksperimen dan Analisis **18**

4.1 Kasus Uji 18
4.2 Hasil Eksperimen 18
4.2.1 Hill-Climbing 18
4.2.2 Simulated Annealing 23
4.2.3 Genetic Algorithm 25
4.2.4 Analisis Perbandingan Algoritma 33
4.2.5 Pembanding Exact pada Dataset Tiny 34
4.2.6 Bonus Multi-Truck 40
4.2.7 Bonus Objective Berbasis ETA 47

5 Kesimpulan **55**

6 Pembagian Tugas Anggota Kelompok **55**

Referensi **55**

A Form Penggunaan Generative AI **56**

B Data Eksperimen **59**

1 Deskripsi Persoalan

1.1 Latar Belakang dan Tujuan

Pemuatan paket pada truk merupakan persoalan optimasi yang melibatkan pemilihan paket dan pengaturan letaknya dalam ruang terbatas. Paket dapat memiliki dimensi, nilai, berat, serta sifat kerapuhan yang berbeda. Pemilihan berdasarkan nilai semata belum tentu menghasilkan susunan yang dapat dimuat, karena penempatan paket juga harus memenuhi batas ruang, kapasitas berat, dan aturan penopangan.

Studi kasus pada tugas besar ini meminta konfigurasi pemuatan paket yang memaksimalkan total nilai muatan sebuah truk [1]. Tidak semua paket harus diangkut. Oleh karena itu, solusi mencakup keputusan paket mana yang dimuat, posisi paket tersebut, dan orientasinya.

Tujuan pengerjaan adalah merancang representasi solusi, menggunakan *local search* dengan menerapkan salah satu varian *hill-climbing*, *simulated annealing* (SA), dan *genetic algorithm* (GA), serta mengevaluasi kualitas solusi, durasi pencarian, dan konsistensi hasilnya. Evaluasi juga mencakup pengaruh jumlah populasi dan banyak iterasi pada GA.

1.2 Definisi Permasalahan dan Data Masukan

Masukan terdiri atas himpunan paket $P = \{1, \ldots, N\}$ dan sebuah truk dengan satu peti kemas. Setiap paket berbentuk balok atau kubus. Atribut masukan dirangkum pada Tabel 1. Satuan dimensi harus konsisten antarentitas.

Tabel 1: Atribut paket dan truk

Entitas	Atribut	Makna
Paket	`id`	Identitas paket.
	`Dimensions`	Lebar, panjang, dan tinggi dasar paket.
	`Orientation`	Pemetaan sisi paket terhadap sumbu koordinat.
	`Value`	Nilai paket dalam satuan unit.
	`Weight`	Berat paket dalam satuan unit berat.
	`IsFragile`	Penanda paket pecah-belah.
	`ETA`	Perkiraan waktu kedatangan dalam sisa hari.
Truk	`Dimensions`	Lebar, panjang, dan tinggi peti kemas.
	`MaxCapacity`	Kapasitas maksimum total berat muatan.

1.3 Ketentuan dan Batasan Persoalan

Ketentuan pemodelan yang menjadi acuan adalah sebagai berikut [1].

1. Ruang lingkup wajib mencakup satu truk dengan satu peti kemas.
2. Paket dapat berbeda ukuran, dan tidak seluruh paket harus dimuat. Paket di luar harus seluruhnya berada di luar. Paket tidak boleh memotong dinding.
3. Posisi paket menggunakan koordinat bilangan bulat. Initial state dibentuk acak.
4. Orientasi menyatakan pemetaan sisi paket ke sumbu. Tersedia enam kemungkinan orientasi. Langkah rotasi neighbor adalah 90° terhadap sumbu x, y, atau z.
5. Paket di dalam tidak boleh melayang.
6. Paket fragile tidak boleh menjadi support paket lain.
7. Jumlah berat paket di dalam tidak boleh melebihi kapasitas truk.
8. Neighbor dibentuk melalui pertukaran, pemindahan, atau rotasi paket. Pemindahan dapat mengubah posisi paket dari dalam ke luar truk maupun sebaliknya.

1.4 Keluaran Program

Objektif wajib adalah memaksimalkan jumlah nilai paket yang berada di dalam peti kemas. Solusi akhir harus memperlihatkan status semua paket, termasuk paket yang tidak dimuat, posisi dan orientasi paket yang dimuat, keadaan ruang peti kemas, dan nilai objektif state. Visualisasi program juga harus dapat menampilkan initial state, final state, dan hasil eksperimen.

2 Perancangan Solusi

2.1 Representasi Paket, Truk, dan State

Ruang peti kemas dinyatakan sebagai $[0, W) \times [0, L) \times [0, H)$ dengan z sebagai arah vertikal. Dimensi dasar paket i adalah (w_i^0, ℓ_i^0, h_i^0), beratnya m_i, nilainya v_i, dan status fragile-nya $r_i \in \{0, 1\}$. Orientasi o_i memetakan dimensi dasar menjadi dimensi efektif (w_i, ℓ_i, h_i).

Posisi sudut kiri-bawah-belakang paket dinotasikan sebagai $\boldsymbol{p}_i = (x_i, y_i, z_i)$. Indikator $a_i(s) = 1$ menyatakan paket i dimuat dalam state s, sedangkan $a_i(s) = 0$ menyatakan paket berada di luar. Notasi indikator ini tidak mengharuskan implementasi memakai field boolean. Struktur data program tetap dapat dipilih bebas.

Representasi harus menyimpan identitas semua paket dan informasi ruang terisi maupun kosong. Paket di luar tetap harus dapat ditampilkan, walaupun hanya paket di dalam yang menyumbang nilai muatan. Dalam notasi laporan, keadaan seluruh koordinat dapat diturunkan dari himpunan volume paket. Implementasi dapat memakai grid eksplisit atau representasi lain yang ekuivalen.

2.2 Pembentukan Initial State

Initial state dibentuk oleh fungsi `build_random_state()` pada `src/domain/packing.py`. Proses dimulai dari `State.all_outside()`, yaitu state yang memuat tepat satu placement untuk setiap paket, tetapi seluruh paket berstatus di luar truk. Urutan paket kemudian diacak menggunakan `random.Random` yang dibuat dari seed konfigurasi. Dengan demikian, kasus, konfigurasi, dan seed yang sama menghasilkan proses inisialisasi yang dapat diulang.

Untuk setiap paket, urutan truk dan enam orientasi diacak. Kandidat koordinat tidak dibangkitkan dari seluruh titik dalam kontainer, melainkan dari himpunan *anchor*: titik $(0, 0, 0)$ serta titik yang berada tepat setelah sisi kanan, sisi belakang, atau sisi atas paket yang telah ditempatkan. Pendekatan ini mengurangi jumlah posisi yang perlu dicoba sekaligus mendorong susunan yang rapat.

Setiap kombinasi truk, orientasi, dan anchor diperiksa oleh `try_place()`. Kandidat diterima hanya jika berada di dalam batas truk, tidak bertumpang tindih, tidak melanggar kapasitas berat, mempunyai support yang sah, dan tidak menjadikan paket fragile sebagai penopang. Percobaan untuk paket tersebut dihentikan setelah ditemukan penempatan feasible pertama. Apabila seluruh kombinasi gagal, paket tetap berada di luar kontainer. Hasil akhir diperiksa kembali sebelum dipakai sebagai initial state pencarian.

Listing 1: Alur pembentukan initial state acak

```
state = State.all_outside(problem)
package_ids = [package.id for package in problem.packages]
rng.shuffle(package_ids)

for package_id in package_ids:
    # Acak truk dan orientasi, lalu coba setiap candidate anchor.
    # State diperbarui hanya jika try_place menghasilkan state feasible.
    ...
return state
```

2.3 Pembentukan Neighbor

Pembentukan *neighbor* dilakukan dengan membangkitkan kandidat *move* melalui `iter_candidate_moves()`. Fungsi tersebut menghasilkan pemindahan paket ke luar truk atau ke setiap *anchor* kandidat, rotasi terhadap sumbu yang menghasilkan dimensi efektif berbeda, serta pertukaran untuk setiap pasangan paket. Setiap kandidat kemudian diterapkan oleh `apply_move()`. Pemindahan dan rotasi diperiksa melalui `try_place()`, sedangkan pertukaran diperiksa melalui `validate_placement_changes()`. Fungsi `iter_feasible_neighbors()` hanya mengembalikan state hasil yang valid, tidak bernilai `None`, dan berbeda dari state asal. Dengan mekanisme ini, algoritma pencarian hanya mengevaluasi *neighbor* yang memenuhi seluruh kendala pemuatan.

2.4 Fungsi Objektif

Untuk himpunan state valid $\mathcal{F}$, persoalan wajib diformulasikan sebagai

$$\max_{s \in \mathcal{F}} f(s), \qquad f(s) = \sum_{i=1}^{N} a_i(s) v_i. \tag{1}$$

Nilai paket di luar tidak dihitung sebagai nilai muatan. Jika program memakai penalti bagi state tidak valid atau heuristik untuk memilih kandidat, fungsi tersebut harus dibedakan dari $f(s)$. Nilai objektif yang dilaporkan untuk solusi akhir tetap mengikuti Persamaan (1) pada state valid.

2.5 Pemeriksaan Validitas State

2.5.1 Batas Ruang dan Tumpang Tindih

Untuk setiap paket yang dimuat, berlaku

$$0 \le x_i, \quad x_i + w_i \le W, \quad 0 \le y_i, \quad y_i + \ell_i \le L, \quad 0 \le z_i, \quad z_i + h_i \le H. \tag{2}$$

Seluruh koordinat posisi berupa bilangan bulat. Dengan volume setengah terbuka $V_i = [x_i, x_i + w_i) \times [y_i, y_i + \ell_i) \times [z_i, z_i + h_i)$, dua paket yang dimuat tidak boleh memiliki irisan volume: $V_i \cap V_j = \varnothing$ untuk $i \neq j$. Kontak pada permukaan tidak dianggap tumpang tindih volume.

2.5.2 Support dan Paket Melayang

Misalkan proyeksi alas paket i adalah $A_i = [x_i, x_i + w_i) \times [y_i, y_i + \ell_i)$. Paket j menjadi penopang geometris paket i apabila

$$j \in S_i(s) \iff a_j(s) = 1,\ j \neq i,\ z_j + h_j = z_i, \quad A_i \cap A_j \neq \varnothing. \tag{3}$$

Untuk setiap paket di dalam, syarat tidak melayang adalah

$$z_i = 0 \quad \vee \quad S_i(s) \neq \varnothing. \tag{4}$$

Syarat ini mengikuti definisi eksistensi support. Dukungan sebagian alas diperbolehkan. Sentuhan tepi tanpa irisan pada domain alas setengah terbuka tidak dihitung sebagai area penopang. Prosedur pemeriksaan diskret harus konsisten dengan konvensi koordinat yang digunakan.

2.5.3 Larangan Fragile Menjadi Penopang

Larangan fragile dinyatakan secara terpisah dari kecukupan support:

$$\forall i \text{ yang dimuat}, \quad \forall j \in S_i(s), \quad r_j = 0. \tag{5}$$

Artinya, keberadaan penopang non-fragile tidak membenarkan kontak support lain yang berasal dari paket fragile. Paket fragile sendiri tetap boleh diletakkan di lantai atau di atas paket non-fragile, selama tidak menopang paket lain.

2.5.4 Kapasitas Berat

Jika kapasitas maksimum truk adalah C, kendala muatan adalah

$$\sum_{i=1}^{N} a_i(s) m_i \leq C. \tag{6}$$

Kapasitas ini tidak menyatakan jumlah paket atau volume. Kendala ruang dan berat diperiksa secara terpisah.

2.6 Rancangan Algoritma Local Search

2.6.1 Hill-Climbing

Hill-climbing melakukan pencarian dengan memperbaiki konfigurasi saat ini melalui neighbor, tanpa membentuk pohon pencarian. Metode ini dapat berhenti pada local maximum atau plateau sehingga tidak menjamin global optimum [2]. Pada persoalan pemuatan paket, pencarian menggunakan nilai objektif $f(s)$ dan hanya mempertimbangkan neighbor valid yang dibangkitkan oleh operator pertukaran, pemindahan, dan rotasi.

- **Steepest-Ascent Hill-Climbing**
  Setiap iterasi memilih neighbor dengan nilai objektif tertinggi. Perpindahan hanya dilakukan jika nilainya lebih besar daripada state sekarang. Jika tidak ada perbaikan, pencarian dihentikan [2]. Jika beberapa kandidat terbaik bernilai sama, implementasi memilih salah satunya secara acak.
- **Stochastic Hill-Climbing**
  Varian ini memilih secara acak di antara langkah yang meningkatkan nilai objektif. AIMA memperbolehkan peluang pemilihan bergantung pada besarnya perbaikan [2]. Implementasi menggunakan bobot $\Delta(s') = f(s') - f(s) > 0$, sehingga

  $$P(s' \mid s) = \frac{\Delta(s')}{\sum_{u \in N^+(s)} \Delta(u)}, \qquad N^+(s) = \{u \in N(s) \mid f(u) > f(s)\}, \tag{7}$$

  dengan $N(s)$ sebagai himpunan neighbor valid yang dibangkitkan. Pencarian berhenti jika tidak ada langkah membaik atau batas iterasi tercapai.
- **Hill-Climbing with Sideways Move**
  Ketika tidak ada langkah membaik, perpindahan ke neighbor bernilai sama diperbolehkan untuk menelusuri plateau. Jumlah langkah sideways berturut-turut dibatasi agar pencarian tidak berjalan tanpa batas [2]. Implementasi mengutamakan perbaikan, menghindari kunjungan ulang state pada plateau yang sama, dan mereset penghitung sideways setelah nilai objektif meningkat.
- **Random-Restart Hill-Climbing**
  Pencarian diulang dari initial state acak untuk mengeksplorasi bagian ruang keadaan yang berbeda [2]. Implementasi menjalankan steepest-ascent pada setiap percobaan dan mengembalikan state terbaik yang telah dievaluasi. Parameter maksimum restart membatasi jumlah pengulangan setelah percobaan pertama. Karena jumlah restart dan evaluasi dibatasi, hasilnya tetap tidak dijamin optimal secara global.

2.6.2 Simulated Annealing

Simulated annealing (SA) menggunakan satu state aktif dan memperbolehkan perpindahan ke state yang lebih buruk agar pencarian dapat keluar dari optimum lokal. Pada setiap iterasi, satu neighbor feasible s' dipilih secara acak dari $N(s)$. Dengan

$$\Delta E = f(s') - f(s), \tag{8}$$

probabilitas penerimaan yang digunakan adalah

$$P(s' \mid s, T) = \begin{cases} 1, & \Delta E \geq 0, \\ \exp(\Delta E / T), & \Delta E < 0. \end{cases} \tag{9}$$

Dengan demikian, langkah yang memperbaiki atau mempertahankan objective selalu diterima, sedangkan langkah yang memperburuk objective masih dapat diterima dengan peluang yang mengecil ketika temperatur menurun.

Temperatur awal dikalibrasi dari median besar penurunan objective pada neighbor feasible. Jika median penurunan tersebut adalah d, temperatur awal dipilih agar penurunan tipikal memiliki probabilitas penerimaan 0,8:

$$T_0 = \max\left(\frac{-d}{\ln 0{,}8}, 10T_{\min}\right). \tag{10}$$

Jika tidak ditemukan penurunan pada sampel kalibrasi, digunakan nilai $\max(1, 10T_{\min})$. Jadwal pendinginan bersifat geometrik,

$$T_k = \max(T_{\min}, T_0 \alpha^k), \qquad \alpha = \left(\frac{T_{\min}}{T_0}\right)^{1/K}, \tag{11}$$

dengan K dibatasi oleh maksimum iterasi dan sisa anggaran evaluasi.

State terbaik yang pernah dievaluasi disimpan terpisah dari state aktif. Hal ini diperlukan karena state akhir SA dapat lebih buruk daripada state terbaik akibat penerimaan langkah menurun. Kondisi *stuck* didefinisikan sebagai state yang tidak mempunyai neighbor feasible dengan objective lebih tinggi. Agar beberapa iterasi berturut-turut pada kondisi yang sama tidak dihitung berulang sebagai kejadian baru, implementasi hanya mencatat awal setiap episode *stuck*.

2.6.3 Genetic Algorithm

Genetic Algorithm (GA) mencari solusi dengan mengembangkan sekumpulan kandidat secara bersamaan. Kandidat-kandidat tersebut membentuk populasi. Pada setiap generasi, program memilih induk, membentuk keturunan melalui *crossover* dan mutasi, lalu menggunakan keturunan tersebut untuk membentuk generasi berikutnya. Proses ini dilanjutkan sampai batas generasi atau batas evaluasi tercapai [2].

Pada persoalan pemuatan paket, satu individu direpresentasikan oleh kromosom yang berisi satu *gene* untuk setiap paket. Gene menyimpan prioritas, orientasi paket, dan truk yang dicoba terlebih dahulu. Prioritas berupa angka acak yang menentukan urutan pemrosesan paket. Penggunaan angka sebagai kunci pengurutan dikenal sebagai *random keys* [3]. Posisi gene tetap berkaitan dengan paket yang sama sehingga *crossover* tidak menghilangkan paket atau membuat satu paket muncul lebih dari sekali.

Kromosom tidak menyimpan koordinat paket. Koordinat baru ditentukan ketika kromosom diterjemahkan menjadi state. Decoder mengurutkan paket berdasarkan prioritas, kemudian mencoba menempatkan setiap paket pada *anchor* yang memenuhi kendala. Truk tujuan pada gene dicoba terlebih dahulu, lalu truk lain dapat digunakan sebagai cadangan. Paket yang tidak

dapat ditempatkan tetap berada di luar truk. Dengan cara ini, proses evolusi mengubah urutan, orientasi, dan pilihan truk, sedangkan decoder menentukan posisi fisiknya.

Kualitas individu dihitung dari state hasil decoding menggunakan fungsi objektif yang dipilih. Pada objektif *total value*, fitness merupakan jumlah nilai paket yang berhasil dimuat. Jika objektif *value per urgency* dipilih, fitness mengikuti nilai objektif tersebut. Fitness yang lebih tinggi menunjukkan solusi yang lebih baik. State awal yang dilaporkan berasal dari individu pertama pada populasi awal, bukan dari individu terbaik pada populasi tersebut.

Induk dipilih dengan *roulette-wheel selection*. Peluang terpilih sebanding dengan fitness sehingga individu bernilai tinggi lebih mungkin menjadi induk, tetapi individu lain tetap mempunyai kesempatan untuk dipilih. Jika seluruh fitness bernilai nol, pemilihan dilakukan secara acak dengan peluang yang sama. *Crossover* menggunakan satu titik potong untuk membentuk satu anak dan dijalankan dengan peluang bawaan 0,9. Jika crossover tidak dilakukan, kromosom induk pertama digunakan sebagai calon anak.

Mutasi diterapkan pada setiap anak dengan peluang bawaan 0,2. Ketika mutasi terjadi, program memilih salah satu operasi berikut.

- **Swap.**
  Operasi ini menukar prioritas dan truk tujuan dua paket tanpa menukar orientasi yang melekat pada paket masing-masing.
- **Rotate.**
  Operasi ini mengubah orientasi satu paket apabila rotasi menghasilkan dimensi efektif yang berbeda.
- **Reassign.**
  Operasi ini mengubah prioritas dan truk tujuan satu paket. Decoder tetap dapat mencoba truk lain apabila paket tidak dapat ditempatkan pada truk tujuan.

Secara bawaan, individu terbaik tidak disalin langsung ke generasi berikutnya karena `elite_count` bernilai nol. Oleh sebab itu, individu terbaik pada populasi terakhir dapat berbeda dari individu terbaik yang pernah ditemukan selama pencarian. Program menyimpan keduanya sebagai *final state* dan *best state*. GA tidak menjamin tercapainya optimum global sehingga kualitas solusi tetap perlu dinilai dari hasil run dan eksperimen.

Eksperimen menguji pengaruh ukuran populasi dan jumlah generasi secara terpisah. Sweep ukuran populasi menggunakan nilai 20, 50, dan 100 dengan jumlah generasi tetap 100. Sweep jumlah generasi menggunakan nilai 50, 100, dan 200 dengan ukuran populasi tetap 50. Setiap konfigurasi dijalankan dengan seed 42, 137, dan 2026 sehingga perbandingan tidak bergantung pada satu urutan acak saja.

2.7 Format Input dan Syarat Validitas

Program menerima persoalan melalui editor data pada antarmuka web atau berkas JSON. Objek tingkat atas wajib mempunyai array `trucks` dan `packages`. Contoh struktur minimal ditunjukkan berikut.

Listing 2: Contoh format input JSON

```
{
  "trucks": [{
    "id": "T1",
    "dimensions": {"width": 8, "length": 6, "height": 4},
    "max_capacity": 30
  }],
  "packages": [{
    "id": "P1",
    "dimensions": {"width": 4, "length": 3, "height": 2},
    "value": 32,
    "weight": 10,
```

```
    "is_fragile": false,
    "eta": 3
  }]
}
```

Parsing dilakukan menggunakan model Pydantic pada `src/problem_io/problem_json.py` dengan mode `strict` dan `extra=forbid`. Syarat validitas data masukan adalah sebagai berikut.

1. Terdapat sedikitnya satu truk dan satu paket.
2. Setiap ID tidak kosong, telah dihapus spasi pada awal dan akhirnya, serta unik di dalam jenis entitas yang sama.
3. Lebar, panjang, dan tinggi merupakan bilangan bulat positif.
4. `max_capacity` harus positif dan bernilai hingga.
5. `value` dan `weight` tidak negatif serta bernilai hingga.
6. `is_fragile` harus berupa boolean dan `eta` berupa bilangan bulat tidak negatif.
7. Field yang tidak didefinisikan oleh skema ditolak untuk mencegah salah eja atau data yang tidak pernah digunakan.

Paket yang lebih besar daripada seluruh truk atau lebih berat daripada kapasitas bukan input yang salah. Paket tersebut tetap merupakan bagian dari persoalan dan dapat dipertahankan di luar kontainer. Setelah struktur input valid, endpoint `/api/problems/validate` membentuk preview initial state dan menjalankan validator constraint sebelum tombol eksekusi dapat digunakan.

3 Implementasi Program

3.1 Lingkungan dan Struktur Program

Komponen teknologi dan pembagian tanggung jawab program dirangkum pada dua tabel berikut.

Tabel 2: Tech stack program

Lapisan	Komponen	Peran
Runtime	Python 3.12 atau lebih baru	Menjalankan domain persoalan, algoritma local search, eksperimen, analisis, dan backend.
Backend	FastAPI	Menyediakan REST API untuk validasi problem, eksekusi satu run, eksperimen, job, dan pembanding solusi.
Validasi	Pydantic	Memvalidasi skema JSON secara ketat sebelum data diterjemahkan menjadi objek domain.
Server	Uvicorn	Menjalankan aplikasi FastAPI sebagai server ASGI.
Frontend	React 19	Menyusun workspace interaktif untuk mode Algoritma dan Eksperimen.
Bahasa frontend	TypeScript 5.9	Memberikan tipe statis untuk konfigurasi, state, hasil run, job, dan respons API.
Build frontend	Vite 7.3	Menjalankan server pengembangan, meneruskan request `/api`, dan membangun aset produksi.
Navigasi	React Router 7.9	Menempatkan workspace utama di dalam shell aplikasi dan menangani fallback rute.
Analisis artefak	Matplotlib dan Pandas	Membentuk grafik, tabel rekap, serta keluaran CSV dan JSON eksperimen.
Manajemen dependensi	`uv` dan `npm`	Mengelola dependensi Python dan frontend.

Tabel 3: Struktur program dan tanggung jawabnya

Direktori	Tanggung jawab
`src/domain`	Model data, fungsi objektif, validasi constraint, dan pembentukan packing awal.
`src/search`	Representasi move, pembentukan neighbor, konfigurasi run, trace, anggaran evaluasi, dan dispatch algoritma.
`src/algorithms`	Implementasi Hill-Climbing, Simulated Annealing, dan Genetic Algorithm.
`src/execution`	Eksekusi satu run, pengukuran durasi, perhitungan metrik, dan penyimpanan artefak.
`src/experiments`	Pembentukan matriks eksperimen, eksekusi batch, agregasi, dan rekap hasil.
`src/problem_io`	Parsing, validasi, dan serialisasi format JSON persoalan.
`src/webapi`	Endpoint FastAPI, pengelolaan job asynchronous, dan penyajian aset frontend produksi.
`frontend/src`	Halaman React, komponen editor, viewer state, grafik, penyimpanan lokal, dan komunikasi API.
`tests`	Tiga dataset kasus uji dan data negatif untuk pemeriksaan validasi input.

3.2 Initial State, Neighbor, dan Fungsi Objektif

Initial state bagi algoritma berbasis satu state dibentuk oleh `build_random_state()`. Urutan paket, truk, orientasi, dan anchor diacak menggunakan generator yang berasal dari seed konfigurasi. Setiap percobaan penempatan melewati `try_place()`, sehingga state hanya diperbarui ketika posisi baru memenuhi seluruh constraint. Random-Restart Hill-Climbing memakai prosedur yang sama untuk membentuk state baru pada setiap restart, sedangkan GA membentuk initial state melalui decoding kromosom.

Listing 3: Pembentukan initial state acak

```python
def build_random_state(problem: Problem, rng: random.Random) -> State:
    state = State.all_outside(problem)
    package_ids = [package.id for package in problem.packages]
    rng.shuffle(package_ids)

    for package_id in package_ids:
        trucks = list(problem.trucks)
        orientations = list(Orientation)
        rng.shuffle(trucks)
        rng.shuffle(orientations)
        placed: State | None = None
        for truck in trucks:
            anchors = list(candidate_anchors(problem, state, truck.id))
            rng.shuffle(anchors)
            for orientation in orientations:
                for anchor in anchors:
                    placed = try_place(
                        problem, state, package_id, truck.id, anchor, orientation
                    )
                    if placed is not None:
                        break
                if placed is not None:
                    break
            if placed is not None:
                break
        if placed is not None:
            state = placed
    return state
```

Neighbor dibangkitkan secara *lazy*. Fungsi `iter_candidate_moves()` menghasilkan kandidat `Relocate`, `Rotate`, dan `Swap`. Kandidat tersebut diterapkan satu per satu oleh `apply_move()`.

Iterator hanya meneruskan state yang valid dan benar- benar berbeda dari state asal sehingga algoritma tidak perlu menyimpan seluruh neighbor di memori.

Listing 4: Pembentukan kandidat move dan neighbor feasible

```python
def iter_candidate_moves(problem: Problem, state: State) -> Iterator[Move]:
    for placement in state.placements:
        if not placement.is_outside:
            yield Relocate(placement.package_id, None, None)

        state_without_package = state.replace(
            Placement(placement.package_id, orientation=placement.orientation)
        )
        for truck in problem.trucks:
            for anchor in candidate_anchors(problem, state_without_package, truck.id):
                if placement.truck_id != truck.id or placement.position != anchor:
                    yield Relocate(placement.package_id, truck.id, anchor)

        current_dimensions = problem.package(placement.package_id).dimensions.oriented(
            placement.orientation
        )
        yielded_dimensions = {
            (
                current_dimensions.width,
                current_dimensions.length,
                current_dimensions.height,
            )
        }
        for axis in ("x", "y", "z"):
            orientation = rotated_orientation(problem, placement, axis)
            dimensions = problem.package(placement.package_id).dimensions.oriented(
                orientation
            )
            key = (dimensions.width, dimensions.length, dimensions.height)
            if key not in yielded_dimensions:
                yielded_dimensions.add(key)
                yield Rotate(placement.package_id, axis)

    package_ids = [placement.package_id for placement in state.placements]
    for index, package_a in enumerate(package_ids):
        for package_b in package_ids[index + 1 :]:
            yield Swap(package_a, package_b)

def iter_feasible_neighbors(
    problem: Problem,
    state: State,
) -> Iterator[tuple[Move, State]]:
    for move in iter_candidate_moves(problem, state):
        successor = apply_move(problem, state, move)
        if successor is not None and successor != state:
            yield move, successor
```

Fungsi objektif menghitung nilai hanya dari paket yang berada di dalam truk. Konfigurasi `TOTAL_VALUE` menjumlahkan nilai paket, sedangkan `VALUE_PER_URGENCY` membagi nilai setiap paket dengan ETA+1. `RunContext.evaluate()` memanggil fungsi ini, mengurangi sisa anggaran evaluasi, serta memperbarui best state ketika nilai yang baru lebih tinggi.

Listing 5: Perhitungan fungsi objektif

```python
def evaluate(problem: Problem, state: State, objective: ObjectiveKind) -> float:
    inside = (
        problem.package(placement.package_id)
        for placement in state.placements
        if not placement.is_outside
    )
```

```
    if objective is ObjectiveKind.TOTAL_VALUE:
        return sum(package.value for package in inside)
    return sum(package.value / (package.eta + 1) for package in inside)
```

Listing 6: Pengelolaan evaluasi dan best state

```
def evaluate(self, state: State, *, track_best: bool = True) -> float:
    self.check_cancelled()
    self.check_budget(1)
    value = evaluate(self.problem, state, self.config.objective)
    self.evaluations += 1
    if track_best and value > self.best_objective:
        self.best_objective = value
        self.best_state = state
    return value
```

3.3 Implementasi Hill-Climbing

Keempat varian ditempatkan pada direktori `src/algorithms/hill_climbing/`. Setiap fungsi menerima `Problem`, `RunConfig`, serta initial state dan callback kemajuan opsional, lalu menghasilkan `SearchResult`. Fungsi `create_run_context()` membentuk `RunContext` dengan seed yang ditetapkan, sedangkan `resolve_initial_state()` membangkitkan state acak jika tidak diberikan dan memeriksa validitasnya.

Neighbor disediakan oleh `iter_feasible_neighbors()`. Evaluasi dilakukan melalui `context.evaluate()`, yang menghitung nilai objektif, memakai anggaran `max_evaluations`, dan memperbarui state terbaik yang pernah dievaluasi. Histori disimpan melalui `context.record()` sebagai `TracePoint`. Seluruh varian dapat dihentikan ketika anggaran evaluasi habis atau pencarian dibatalkan. Potongan kode berikut memperlihatkan bagian inti masing-masing varian.

3.3.1 Steepest-Ascent Hill-Climbing

Listing 7: Pemilihan neighbor terbaik pada steepest-ascent

```
selected_state: State | None = None
selected_objective = current_objective
tie_count = 0

for _, neighbor in iter_feasible_neighbors(problem, current):
    neighbor_objective = context.evaluate(neighbor)
    comparison = context.compare(
        neighbor_objective,
        selected_objective,
    )
    if comparison > 0:
        selected_state = neighbor
        selected_objective = neighbor_objective
        tie_count = 1
    elif (
        comparison == 0
        and context.is_better(neighbor_objective, current_objective)
    ):
        tie_count += 1
        if context.rng.randrange(tie_count) == 0:
            selected_state = neighbor
```

Fungsi `run_steepest_ascent()` pada `steepest_ascent.py` menyiapkan pencarian dan memanggil `_run_steepest_climb()`. Fungsi internal tersebut mengevaluasi seluruh neighbor yang dibangkitkan sebelum menerima langkah terbaik. Pemilihan acak pada nilai terbaik yang sama dilakukan secara bertahap menggunakan penghitung `tie_count`.

Jika `selected_state` tetap `None`, fungsi mengembalikan state sekarang dengan alasan `CONVERGED`. Jika ada perbaikan, state diperbarui, jumlah iterasi ditambah, dan perubahan objektif dicatat.

3.3.2 Stochastic Hill-Climbing

Listing 8: Pemilihan uphill move berbobot pada stochastic hill-climbing

```
for _, neighbor in iter_feasible_neighbors(problem, current):
    neighbor_objective = context.evaluate(neighbor)
    delta = neighbor_objective - current_objective
    if context.is_better(neighbor_objective, current_objective):
        uphill_moves.append((neighbor, neighbor_objective, delta))

if not uphill_moves:
    break

weights = [delta for _, _, delta in uphill_moves]
current, current_objective, delta = context.rng.choices(
    uphill_moves,
    weights=weights,
    k=1,
)[0]
probability = delta / sum(weights)
```

Fungsi `run_stochastic()` pada `stochastic.py` mengumpulkan seluruh neighbor membaik dalam `uphill_moves`, lalu memakai `rng.choices()` dengan bobot kenaikan objektif. Implementasi ini memilih dari kumpulan uphill moves, bukan mengambil kandidat membaik pertama yang ditemukan.

Setiap langkah terpilih menambah iterasi dan mencatat probabilitas pemilihannya pada `accept_prob`. Pencarian berhenti jika `uphill_moves` kosong atau `max_iterations` tercapai.

3.3.3 Hill-Climbing with Sideways Move

Listing 9: Penerimaan dan pembatasan langkah sideways

```
if sideways_state is None:
    break

if config.max_sideways == 0:
    stopped_reason = StopReason.SIDEWAYS_LIMIT
    break

current = sideways_state
sideways_moves += 1
plateau_visited.add(current)
context.record(
    current_objective,
    delta=0.0,
    accepted=True,
    sideways_moves=sideways_moves,
)

if sideways_moves >= config.max_sideways:
    stopped_reason = StopReason.SIDEWAYS_LIMIT
    break
```

Fungsi `run_sideways()` pada `sideways.py` memilih neighbor membaik terbaik terlebih dahulu. Jika tidak tersedia, fungsi memilih secara acak neighbor bernilai sama yang belum ada dalam `plateau_visited`. Saat terjadi perbaikan, `sideways_moves` direset menjadi nol dan himpunan kunjungan diinisialisasi ulang dengan state baru.

Batas `max_sideways` berlaku untuk langkah sideways berturut-turut. Pencarian langsung berhenti setelah langkah yang mencapai batas tersebut. Nilai nol menonaktifkan langkah sideways.

3.3.4 Random-Restart Hill-Climbing

Listing 10: Pengulangan steepest-ascent dari initial state berbeda

```
for restart in range(config.num_restarts + 1):
    if not context.has_budget():
        stopped_reason = StopReason.BUDGET_EXHAUSTED
        break

    context.rng = random.Random(child_seed(config.seed, restart))
    restart_initial = resolve_initial_state(
        problem,
        context,
        initial_state if restart == 0 else None,
    )
    restart_objective = context.evaluate(restart_initial)

    # ... Pencatatan initial state dan trace ...
    climb = _run_steepest_climb(
        problem,
        context,
        restart_initial,
        restart_objective,
        restart=restart,
    )

    if climb.stopped_reason is not StopReason.CONVERGED:
        stopped_reason = climb.stopped_reason
        break
```

Fungsi `run_random_restart()` pada `random_restart.py` memanggil `_run_steepest_climb()` berulang kali. Setiap percobaan memakai seed turunan dari `child_seed()`. Initial state yang diberikan hanya digunakan pada percobaan pertama. Untuk percobaan berikutnya, memakai state acak baru.

Parameter `num_restarts` menyatakan restart tambahan sehingga maksimum percobaan adalah `num_restarts + 1`. Seluruh percobaan berbagi satu anggaran evaluasi. Field `restart` dan `iteration_in_restart` pada trace mendukung pencatatan iterasi per restart. Hasil akhir diambil dari `context.best_state`, yaitu state terbaik yang telah dievaluasi, bukan selalu state terakhir yang dikunjungi.

3.4 Implementasi Simulated Annealing

Fungsi utama `run_simulated_annealing()` menerima `Problem`, `RunConfig`, initial state opsional, dan callback kemajuan, kemudian menghasilkan `SearchResult`. Jika initial state tidak diberikan, `build_random_state()` membentuk state acak menggunakan seed pada konfigurasi. State tersebut divalidasi sebelum pencarian dimulai.

Pemilihan neighbor menggunakan *reservoir sampling* pada iterator `iter_feasible_neighbors()`, sehingga setiap neighbor feasible mempunyai peluang yang sama untuk terpilih tanpa menyimpan seluruh neighbor di memori. Potongan berikut menunjukkan fungsi probabilitas penerimaan.

Listing 11: Probabilitas penerimaan pada simulated annealing

```
def acceptance_probability(delta: float, temperature: float) -> float:
    if temperature <= 0:
```

```
        raise ValueError("temperature must be positive")
    return 1.0 if delta >= 0 else math.exp(delta / temperature)
```

Temperatur awal dihitung oleh `_initial_temperature()` menggunakan maksimum `calibration_samples` penurunan objective. Setelah itu, `alpha` ditentukan agar temperatur mendekati `t_min` pada akhir langkah yang tersedia. Bagian inti satu iterasi ditunjukkan berikut.

Listing 12: Penerimaan neighbor dan pendinginan temperatur

```
neighbor_objective = context.evaluate(neighbor)
delta = neighbor_objective - current_objective
temperature = max(
    config.t_min,
    temperature_0 * alpha ** (iteration + 1),
)
probability = acceptance_probability(delta, temperature)
random_draw = context.rng.random()
accepted = random_draw < probability

if accepted:
    current = neighbor
    current_objective = neighbor_objective

context.record(
    current_objective,
    temperature=temperature,
    delta=delta,
    accept_prob=probability,
    random_draw=random_draw,
    accepted=accepted,
    stuck=stuck and not stuck_episode,
)
```

`RunContext` mencatat objective saat ini dan terbaik, jumlah evaluasi, durasi, temperatur, ΔE, probabilitas, bilangan acak, keputusan penerimaan, serta episode *stuck*. Pencarian berhenti ketika maksimum iterasi tercapai, anggaran evaluasi habis, tidak terdapat neighbor feasible, atau pengguna membatalkan proses. Hasil yang dikembalikan tetap menyertakan initial state, final state, dan best state sehingga ketiganya dapat dianalisis secara terpisah.

3.5 Implementasi Genetic Algorithm

Implementasi Genetic Algorithm ditempatkan pada direktori `src/algorithms/genetic_algorithm/`. Algoritma ini menggunakan populasi individu sebagai kandidat solusi. Setiap individu menyimpan kromosom, state hasil decoding, dan nilai fitness. Pada setiap generasi, induk dipilih menggunakan *roulette-wheel selection*. Kromosom anak kemudian dibentuk melalui *crossover* dan mutasi sebelum dimasukkan ke populasi generasi berikutnya.

3.5.1 Representasi Kromosom dan Individu

Setiap kromosom berisi satu gene untuk setiap paket. Gene menyimpan nilai prioritas, orientasi paket, dan truk tujuan yang diutamakan. Identitas paket tidak disimpan di dalam gene. Posisi gene pada kromosom sesuai dengan urutan paket pada `problem.packages`. Nilai prioritas menentukan urutan paket diproses oleh decoder.

Listing 13: Representasi kromosom dan individu

```
@dataclass(frozen=True, slots=True)
class Gene:
    priority: float
    orientation: Orientation
    truck_id: str | None
```

```python

Chromosome = tuple[Gene, ...]

@dataclass(frozen=True, slots=True)
class Individual:
    chromosome: Chromosome
    state: State
    fitness: float
```

Fungsi `random_chromosome()` membuat satu gene acak untuk setiap paket. Gene tersebut memiliki prioritas acak, orientasi acak, serta truk tujuan yang dipilih dari truk yang tersedia atau `None`. Nilai `None` berarti gene tidak menetapkan truk tujuan sehingga paket tetap berada di luar truk ketika decoding.

Populasi awal dibentuk dengan membuat dan mengevaluasi sejumlah kromosom sebanyak `config.population_size`. Setiap kromosom diubah menjadi objek `Individual` sebelum dimasukkan ke populasi.

Listing 14: Pembentukan populasi awal

```python
for _ in range(config.population_size):
    individual = make_individual(random_chromosome(problem, rng), {})
    population.append(individual)
    if initial is None:
        initial = individual
```

3.5.2 Decoding dan Evaluasi Fitness

Kromosom tidak menyimpan koordinat fisik paket secara langsung. Kromosom diterjemahkan menjadi `State` oleh fungsi `decode_chromosome()`. Decoder memproses paket menurut prioritas menaik. Untuk setiap paket, decoder mencoba truk yang dituju oleh gene terlebih dahulu, kemudian truk lain sebagai cadangan. Pada truk yang dicoba, decoder memeriksa beberapa *anchor* dan menempatkan paket pada posisi pertama yang memenuhi kendala. Jika paket tidak memiliki truk tujuan atau tidak dapat ditempatkan, paket tetap berada di luar truk.

Listing 15: Decoding kromosom menjadi state

```python
def decode_chromosome(problem: Problem, chromosome: Chromosome) -> State:
    validate_chromosome(problem, chromosome)
    packages = problem.packages
    state = State(
        tuple(
            Placement(package.id, orientation=gene.orientation)
            for package, gene in zip(packages, chromosome)
        )
    )
    used_weight = {truck.id: 0.0 for truck in problem.trucks}

    order = sorted(range(len(packages)), key=lambda i: (chromosome[i].priority, i))
    for index in order:
        package = packages[index]
        gene = chromosome[index]
        if gene.truck_id is None:
            continue
        oriented = package.dimensions.oriented(gene.orientation)
        trucks = sorted(problem.trucks, key=lambda t: t.id != gene.truck_id)
        for truck in trucks:
            if not _can_fit(truck, oriented, package.weight, used_weight[truck.id]):
                continue
            placed = None
            for anchor in candidate_anchors(problem, state, truck.id):
                placed = try_place(
                    problem, state, package.id, truck.id, anchor, gene.orientation
```

```
                )
                if placed is not None:
                    break
            if placed is not None:
                state = placed
                used_weight[truck.id] += package.weight
                break
    return state
```

Pemeriksaan pada `try_place()` memastikan penempatan yang dicoba memenuhi kendala geometri dan aturan pemuatan. Setelah state terbentuk, fitness dihitung menggunakan `RunContext.evaluate()`. Fungsi ini mengevaluasi state berdasarkan objective yang dipilih pada konfigurasi.

Listing 16: Pembentukan dan evaluasi individu

```
def make_individual(
    chromosome: Chromosome, known: dict[Chromosome, Individual]
) -> Individual:
    ctx.check_cancelled()
    ctx.check_budget()
    previous = known.get(chromosome)
    state = previous.state if previous is not None else decode_chromosome(problem, chromosome)
    return Individual(chromosome, state, ctx.evaluate(state))
```

Fitness yang lebih tinggi menunjukkan solusi yang lebih baik berdasarkan objective yang sedang digunakan. Dengan demikian, fitness tidak selalu berarti total nilai paket jika objective yang dipilih berbeda.

3.5.3 Seleksi Induk

Pemilihan induk menggunakan *roulette-wheel selection*. Peluang individu terpilih sebanding dengan fitness-nya. Fitness harus bernilai nonnegatif. Jika seluruh fitness bernilai nol, program memilih individu secara acak dengan peluang yang sama.

Listing 17: Seleksi induk

```
x = roulette_select(population, rng)
y = roulette_select(population, rng)
```

Fungsi `roulette_select()` menerima populasi berisi objek `Individual` dan generator bilangan acak. Fungsi ini mengambil fitness dari setiap individu untuk menentukan pilihan induk.

3.5.4 Crossover

Program menggunakan *crossover* satu titik. Titik potong dipilih secara acak, lalu bagian kromosom sebelum titik tersebut diambil dari induk pertama dan sisanya dari induk kedua. Crossover menghasilkan satu kromosom anak. Karena posisi gene selalu mewakili paket yang sama, penggabungan segmen ini tidak menghilangkan atau menggandakan identitas paket.

Peluang crossover diatur melalui `config.crossover_prob`. Jika crossover tidak dilakukan, kromosom induk pertama digunakan sebagai kromosom anak.

Listing 18: Crossover dan pemilihan kromosom anak

```
if rng.random() < config.crossover_prob:
    child = crossover(x.chromosome, y.chromosome, rng)
else:
    child = x.chromosome
```

3.5.5 Mutasi

Setelah kromosom anak dibentuk, program memeriksa peluang mutasi. Jika mutasi terjadi, fungsi `mutate_chromosome()` memilih satu gene dan salah satu operasi mutasi berikut.

- **Swap.**
  Operasi *swap* menukar prioritas dan truk tujuan antara dua gene, tetapi orientasi tetap melekat pada paket masing-masing. Operasi ini hanya tersedia jika kromosom berisi lebih dari satu gene.
- **Rotate.**
  Operasi *rotate* mengubah orientasi satu paket. Operasi ini hanya tersedia jika rotasi menghasilkan orientasi dengan dimensi berbeda.
- **Reassign.**
  Operasi *reassign* mengubah prioritas dan truk tujuan satu paket. Truk tujuan tersebut tetap merupakan truk yang dicoba terlebih dahulu oleh decoder. Decoder masih dapat mencoba truk lain sebagai cadangan.

Listing 19: Penerapan mutasi pada kromosom anak

```
if rng.random() < mutation_prob:
    child = mutate_chromosome(problem, child, rng)
```

Jika `config.mutation_prob` tidak diberikan, peluang mutasi bawaan ditetapkan oleh `DEFAULT_MUTATION_PROB` dengan nilai 0,2. Peluang ini diterapkan pada setiap anak, bukan secara terpisah pada setiap gene.

3.5.6 Pembentukan Generasi dan Penghentian

Populasi generasi berikutnya dibentuk dengan memilih induk, menerapkan crossover sesuai peluangnya, lalu menerapkan mutasi sesuai peluang mutasi. Proses ini berulang sampai ukuran populasi terpenuhi. Secara bawaan, elitisme tidak digunakan karena nilai `GAOptions.elite_count` adalah nol.

Listing 20: Pembentukan populasi generasi berikutnya

```
known = {individual.chromosome: individual for individual in population}
ranked = sorted(population, key=lambda item: item.fitness, reverse=True)
next_population = ranked[: options.elite_count]

while len(next_population) < config.population_size:
    x = roulette_select(population, rng)
    y = roulette_select(population, rng)
    if rng.random() < config.crossover_prob:
        child = crossover(x.chromosome, y.chromosome, rng)
    else:
        child = x.chromosome
    if rng.random() < mutation_prob:
        child = mutate_chromosome(problem, child, rng)
    next_population.append(make_individual(child, known))

population = next_population
```

Pencarian berhenti setelah jumlah generasi pada `config.generations` tercapai. Pencarian juga dapat berhenti jika anggaran evaluasi habis atau jika proses dibatalkan. Pengelolaan batas evaluasi dan pembatalan dilakukan melalui `RunContext`.

3.5.7 Pencatatan Hasil Pencarian

Pada setiap generasi, program mencatat fitness maksimum dan rata-rata fitness populasi. Pencatatan dilakukan melalui `ctx.record()` dan menghasilkan data yang dapat digunakan

untuk membuat grafik terhadap jumlah generasi.

Listing 21: Pencatatan fitness populasi

```python
def record(population: list[Individual], generation: int) -> None:
    fitnesses = [individual.fitness for individual in population]
    ctx.record(
        max(fitnesses),
        generation=generation,
        population_mean_objective=sum(fitnesses) / len(fitnesses),
    )
```

State awal adalah state hasil decoding individu pertama pada populasi awal. *Final state* adalah state dengan fitness tertinggi pada populasi terakhir, sedangkan *best state* adalah state dengan fitness tertinggi yang ditemukan selama seluruh pencarian. Karena elitisme tidak digunakan secara bawaan, *final state* dapat berbeda dari *best state*.

Eksperimen GA menguji pengaruh ukuran populasi dan jumlah generasi secara terpisah. Protokol sweep populasi menggunakan ukuran populasi 20, 50, dan 100 dengan jumlah generasi tetap 100. Protokol sweep generasi menggunakan populasi tetap 50 dengan jumlah generasi 50, 100, dan 200. Setiap konfigurasi dijalankan untuk setiap seed yang dipilih. Hasil run menyimpan state awal, state terbaik, state akhir, nilai objective, durasi, serta data trajectory populasi.

3.6 Visualisasi dan Penyimpanan Hasil

Frontend dibangun sebagai aplikasi React satu halaman. Rute aktif menampilkan `SetupPage` di dalam `Shell`. Halaman ini menyediakan dua mode kerja, yaitu Algoritma dan Eksperimen. Mode Algoritma memuat panel konfigurasi, editor data dalam dialog, viewer tiga dimensi, ringkasan hasil, grafik trajectory, detail run, serta pembanding exact optimum atau knapsack upper bound. Viewer dapat berpindah antara initial state, best state, dan final state. Mode Eksperimen dirender oleh `ExperimentWorkspace` dan membagi alur menjadi Input Algoritma, Kasus Uji, dan Hasil Percobaan. Tampilan hasil eksperimen memuat ringkasan jumlah run, grafik median objective dan durasi, tabel agregat, detail setiap run, serta tombol unduh CSV dan JSON.

Komunikasi frontend dengan algoritma local search berlangsung melalui fungsi pada `frontend/src/api.ts`. Data problem dimuat dari endpoint `/api/problems/example` atau `/api/problems/fixtures/<name>` dan diperiksa melalui `/api/problems/validate`. Saat pengguna menjalankan algoritma, frontend mengirim problem dan konfigurasi ke `/api/runs`. FastAPI menerjemahkan request menjadi objek domain, lalu `execute_run()` meneruskan konfigurasi ke implementasi algoritma yang dipilih. Backend mengembalikan job ID karena proses berjalan secara asynchronous. Frontend kemudian melakukan polling ke `/api/jobs/<id>` setiap 700 milidetik untuk menampilkan progres dan hasil. Eksperimen memakai pola yang sama melalui `/api/experiments/preview`, `/api/experiments`, dan polling job setiap 900 milidetik.

Jika opsi **Simpan Hasil** aktif, backend menyimpan artefak satu run pada `output/runs/<timestamp>/`. Artefak tersebut mencakup hasil JSON, konfigurasi, metrik, trajectory CSV, diagram objective, serta snapshot initial state dan best state. Hasil eksperimen disimpan pada `output/experiments/<timestamp>/` bersama rekap CSV, JSON, grafik median, dan artefak setiap run. Pada mode produksi, FastAPI juga menyajikan hasil build Vite dari `frontend/dist`, sehingga antarmuka dan REST API tersedia dari server yang sama.

4 Eksperimen dan Analisis

4.1 Kasus Uji

Eksperimen menggunakan tiga dataset karena masing-masing mewakili tingkat dan bentuk kesulitan yang berbeda. `tiny_exact.json` berisi satu truk dan enam paket sehingga optimum global dapat dihitung secara eksak dan dipakai sebagai pembanding kualitas solusi. `single_truck_constrained.json` berisi satu truk dan 12 paket serta menjadi kasus utama untuk membandingkan seluruh algoritma, menguji pengaruh seed, dan melakukan sweep ukuran populasi serta jumlah generasi GA. `multi_truck_constrained.json` berisi tiga truk dan 12 paket untuk menguji distribusi paket pada lebih dari satu kendaraan. Ketiga kasus juga memuat paket yang tidak dapat dimasukkan sehingga program harus mempertahankannya di luar truk tanpa menghasilkan state tidak valid. Setiap konfigurasi dijalankan dengan seed 42, 137, dan 2026 menggunakan objective `total_value`. Pengujian bonus pada dataset single-truck menggunakan `value_per_urgency`. Secara keseluruhan, 87 run dipilih untuk menguji kualitas objective, validitas initial state dan best state, konsistensi antarseed, biaya evaluasi, durasi, serta pengaruh parameter GA.

4.2 Hasil Eksperimen

4.2.1 Hill-Climbing

Tabel 4 memuat hasil eksperimen empat varian Hill-Climbing pada dataset single-truck. Nilai $f(s_0)$, $f(s_f)$, dan $f(s^*)$ masing-masing menyatakan objective state awal, state akhir, dan state terbaik yang pernah dievaluasi. Durasi hanya mencakup pencarian, tidak mencakup pembuatan PNG dan penulisan artefak.

Tabel 4: Hasil eksperimen Hill-Climbing pada dataset single-truck.

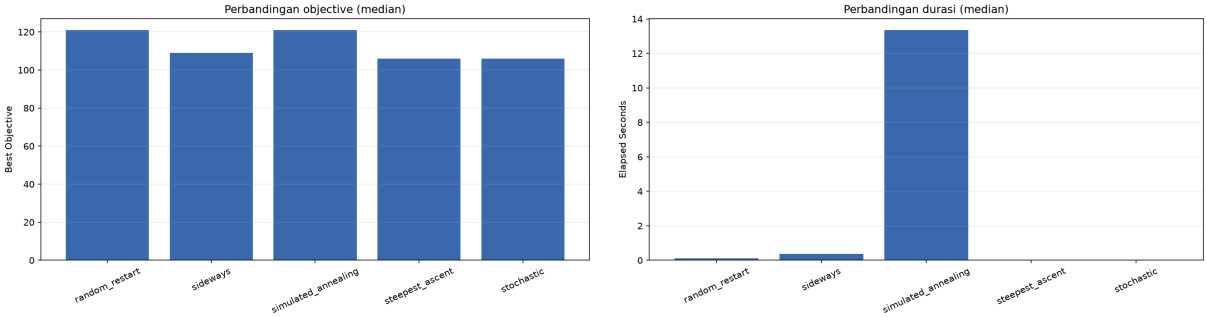
Algoritma	Seed	$f(s_0)$	$f(s_f)$	$f(s^*)$	Iterasi	Evaluasi	Durasi (s)	Berhenti
Steepest	42	112	113	113	1	101	0,0131	converged
Steepest	137	106	106	106	0	63	0,0049	converged
Steepest	2026	103	104	104	1	131	0,0086	converged
Sideways	42	112	117	117	108	6.087	0,5508	sideways limit
Sideways	137	106	109	109	111	6.166	0,3784	sideways limit
Sideways	2026	103	109	109	103	5.936	0,3709	sideways limit
Stochastic	42	112	113	113	1	101	0,0121	converged
Stochastic	137	106	106	106	0	63	0,0047	converged
Stochastic	2026	103	104	104	1	131	0,0078	converged
Random restart	42	104	121	121	6	856	0,1161	restart limit
Random restart	137	112	116	116	6	914	0,0942	restart limit
Random restart	2026	106	121	121	7	885	0,1125	restart limit

Steepest-ascent dan stochastic menghasilkan nilai yang sama pada ketiga seed. Pada seed 137, initial state telah menjadi local optimum sehingga tidak terjadi langkah perbaikan. Pada seed 42 dan 2026, keduanya hanya melakukan satu langkah membaik. Stochastic memilih tepat satu langkah membaik dengan probabilitas seleksi 1,0 pada dua seed tersebut. Dengan hanya satu kandidat yang terpilih, mekanisme stokastik belum menghasilkan lintasan berbeda dari steepest-ascent.

Varian sideways mencapai nilai 117, 109, dan 109. Masing-masing melakukan 106, 110, dan 101 langkah sideways, sedangkan panjang maksimum rangkaian sideways mencapai batas 100 pada semua seed. Nilai terbaiknya melampaui steepest-ascent pada ketiga seed, tetapi tambahan eksplorasi menaikkan median evaluasi dari 101 menjadi 6.087.

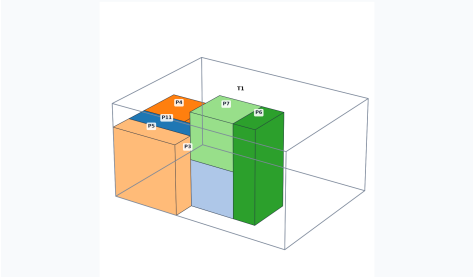
Random-restart menjalankan 11 percobaan per seed, yaitu satu percobaan awal dan 10 restart.

Best restart terjadi pada percobaan ke-2, ke-4, dan ke-10 untuk seed 42, 137, dan 2026. Dua dari tiga run mencapai nilai 121. Seed 137 berhenti pada nilai 116. Hasil ini menunjukkan manfaat diversifikasi initial state, namun juga menunjukkan bahwa jumlah restart terbatas tidak menjamin setiap seed menemukan nilai terbaik yang sama.



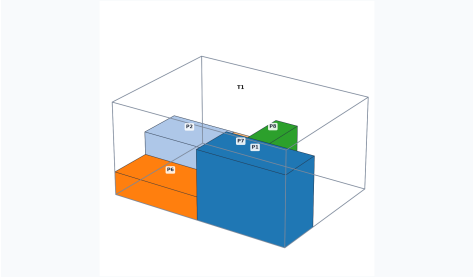
Gambar 1: Median objective terbaik dan durasi lima algoritma non-GA pada dataset single-truck.

Pasangan initial state dan best state untuk seluruh eksperimen Hill-Climbing pada kasus uji utama ditampilkan berikut ini.



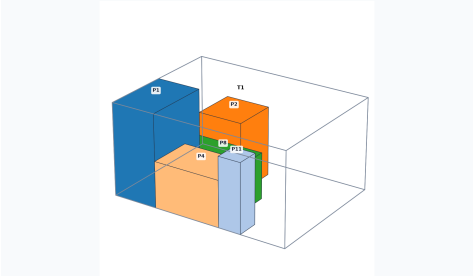
Initial state *Best state*

Gambar 2: Steepest-ascent HC, seed 42, $f(s_0) = 112$ dan $f(s^*) = 113$.



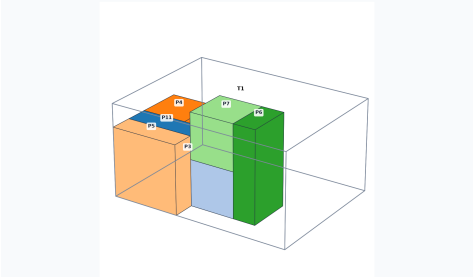
Initial state *Best state*

Gambar 3: Steepest-ascent HC, seed 137, $f(s_0) = 106$ dan $f(s^*) = 106$.

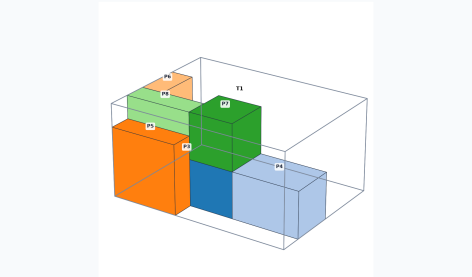


Initial state *Best state*

Gambar 4: Steepest-ascent HC, seed 2026, $f(s_0) = 103$ dan $f(s^*) = 104$.

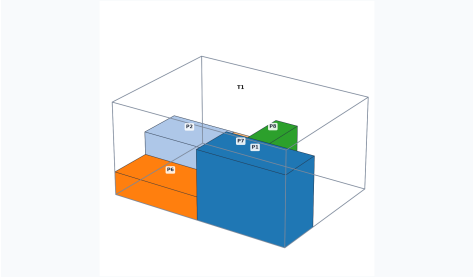


Initial state

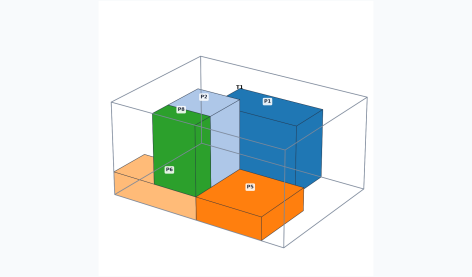


Best state

Gambar 5: HC sideways, seed 42, $f(s_0) = 112$ dan $f(s^*) = 117$.

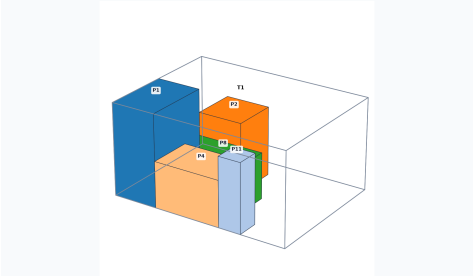


Initial state

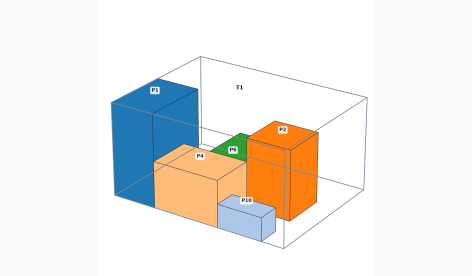


Best state

Gambar 6: HC sideways, seed 137, $f(s_0) = 106$ dan $f(s^*) = 109$.

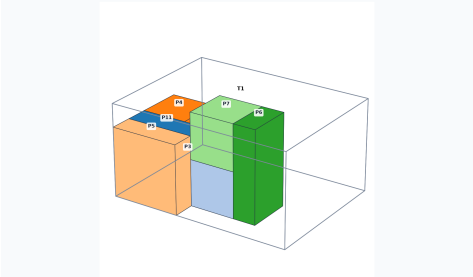


Initial state

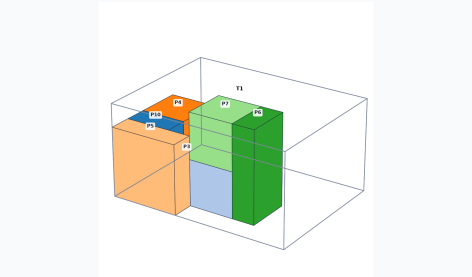


Best state

Gambar 7: HC sideways, seed 2026, $f(s_0) = 103$ dan $f(s^*) = 109$.

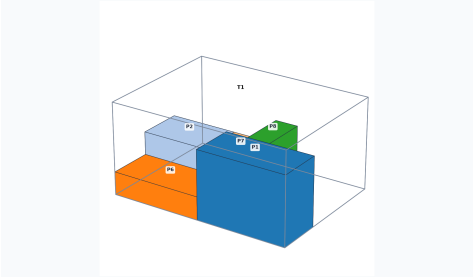


Initial state

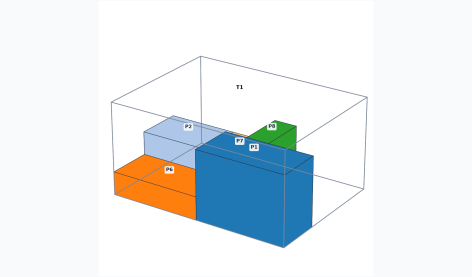


Best state

Gambar 8: Stochastic HC, seed 42, $f(s_0) = 112$ dan $f(s^*) = 113$.

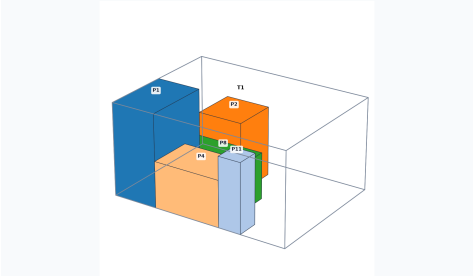


Initial state

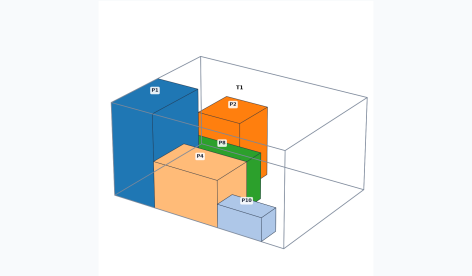


Best state

Gambar 9: Stochastic HC, seed 137, $f(s_0) = 106$ dan $f(s^*) = 106$.

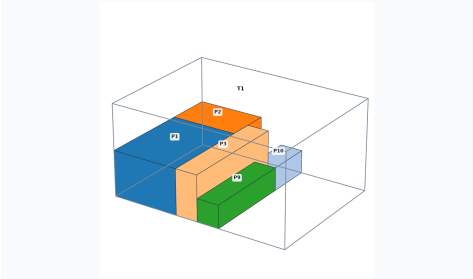


Initial state

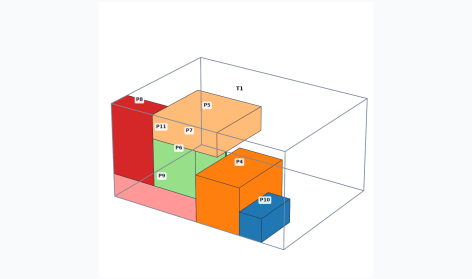


Best state

Gambar 10: Stochastic HC, seed 2026, $f(s_0) = 103$ dan $f(s^*) = 104$.

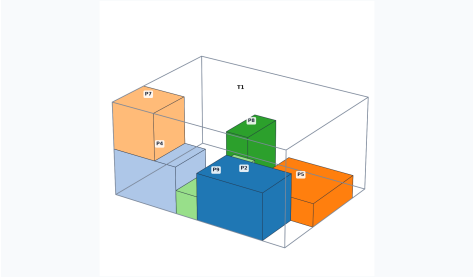


Initial state

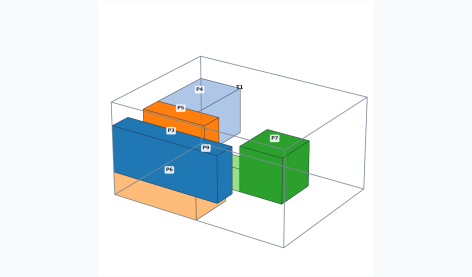


Best state

Gambar 11: Random-restart HC, seed 42, $f(s_0) = 104$ dan $f(s^*) = 121$.

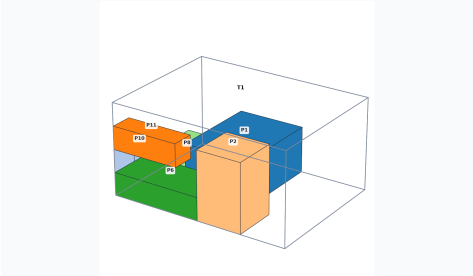


Initial state

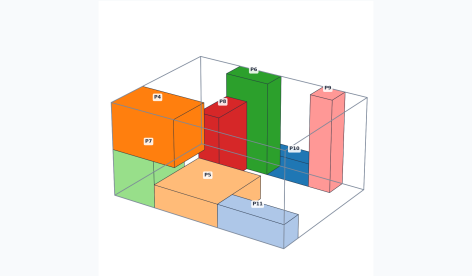


Best state

Gambar 12: Random-restart HC, seed 137, $f(s_0) = 112$ dan $f(s^*) = 116$.



Initial state



Best state

Gambar 13: Random-restart HC, seed 2026, $f(s_0) = 106$ dan $f(s^*) = 121$.

4.2.2 Simulated Annealing

SA menggunakan batas 50.000 evaluasi, maksimum 10.000 iterasi, $T_{\min} = 0{,}001$, dan 100 sampel kalibrasi temperatur. Ketiga run mencapai best objective 121, tetapi final state tidak selalu sama dengan best state. Pemisahan tersebut penting karena SA dapat menerima langkah yang menurunkan objective setelah solusi terbaik ditemukan.

Tabel 5: Hasil eksperimen simulated annealing.

Seed	$f(s_0)$	$f(s_f)$	$f(s^*)$	Iterasi	Evaluasi	Durasi (s)	Stuck
42	112	112	121	1.455	50.000	13,352	49
137	106	121	121	1.277	50.000	12,239	54
2026	103	114	121	1.360	50.000	13,518	51
Rata-rata	107,0	115,7	121,0	1.364	50.000	13,036	51,3

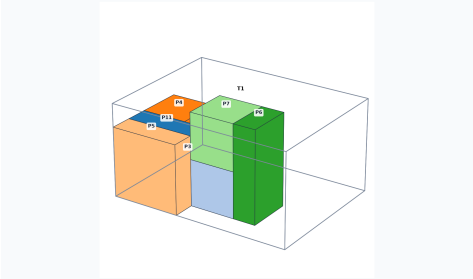
Solusi bernilai 121 yang ditemukan SA memuat P4, P5, P6, P7, P8, P9, P10, dan P11 dengan total berat tepat 30. Hanya seed 137 berakhir pada objective 121. Seed 42 dan 2026 berakhir pada 112 dan 114. Hal tersebut bukan kegagalan pencatatan karena program mengembalikan best state yang dipelihara terpisah.

Tabel 6: Perilaku penerimaan dan temperatur simulated annealing.

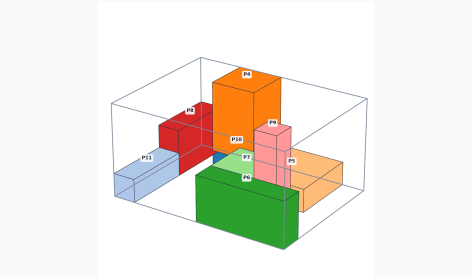
Seed	Diterima	Ditolak	Downhill diterima	T akhir	Stuck/iterasi
42	1.369	86	151	10,233	3,37%
137	1.204	73	123	12,894	4,23%
2026	1.284	76	137	13,966	3,75%

Rasio penerimaan seluruh keputusan berada pada rentang 94,09–94,41%. Sebanyak 123–151 langkah menurun tetap diterima. Semua run berhenti karena anggaran evaluasi habis sebelum mencapai 10.000 iterasi atau temperatur minimum. Temperatur akhir masih 10,233–13,966, sehingga konfigurasi ini menghabiskan banyak evaluasi ketika membentuk dan memeriksa neighbor sebelum memasuki fase temperatur rendah. Best objective 121 setara dengan 83,45% dari upper bound 145, tetapi persentase tersebut bukan gap terhadap optimum global karena upper bound mengabaikan constraint geometris.

Pasangan initial state dan best state untuk seluruh eksperimen simulated annealing pada kasus uji utama ditampilkan berikut ini.

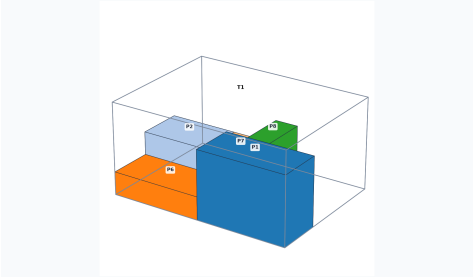


Initial state

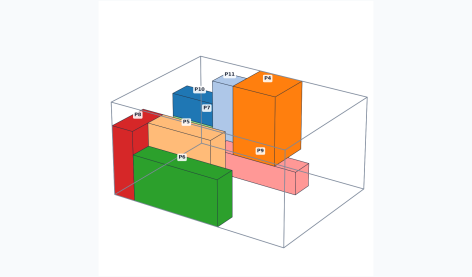


Best state

Gambar 14: Simulated annealing, seed 42, $f(s_0) = 112$ dan $f(s^*) = 121$.

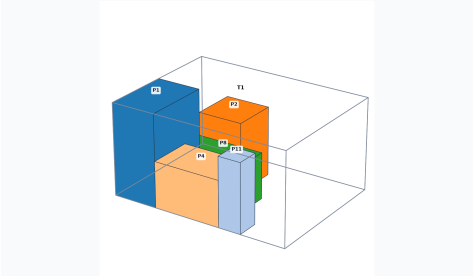


Initial state

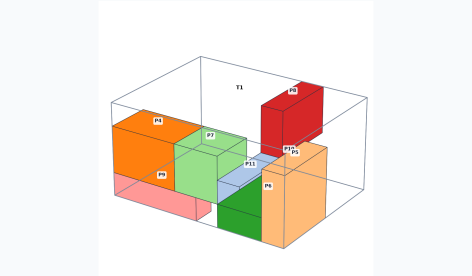


Best state

Gambar 15: Simulated annealing, seed 137, $f(s_0) = 106$ dan $f(s^*) = 121$.



Initial state



Best state

Gambar 16: Simulated annealing, seed 2026, $f(s_0) = 103$ dan $f(s^*) = 121$.

4.2.3 Genetic Algorithm

Pada GA, $f(s_0)$ adalah fitness kromosom pertama yang dievaluasi, bukan fitness terbaik populasi awal. Final state adalah individu terbaik pada populasi terakhir, sedangkan best state adalah individu terbaik sepanjang seluruh generasi. Karena *elite count* bernilai nol, individu terbaik dapat hilang dari populasi berikutnya. Oleh sebab itu $f(s_f)$ dapat lebih rendah daripada $f(s^*)$.

Tabel 7: Sweep jumlah generasi GA dengan populasi tetap 50.

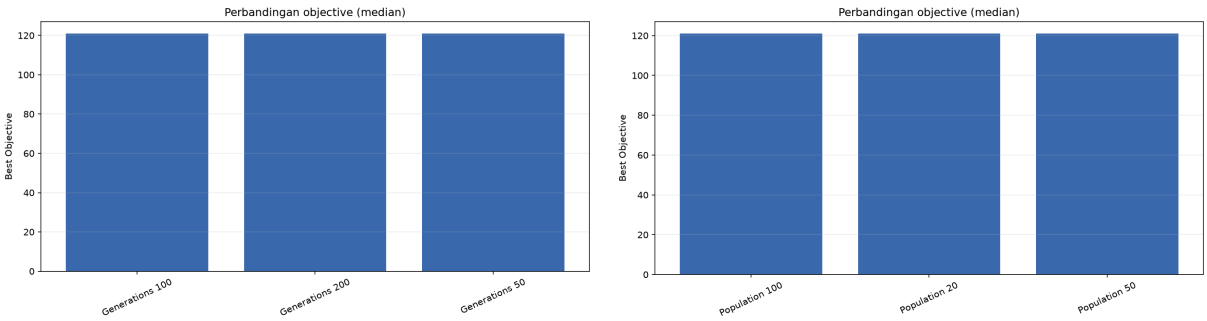
G	Seed	$f(s_0)$	$f(s_f)$	$f(s^*)$	Gen. terbaik	Evaluasi	Durasi (s)	Rerata pop. akhir
50	42	80	116	121	1	2.550	0,766	98,64
50	137	61	118	121	4	2.550	0,961	108,42
50	2026	75	118	121	0	2.550	0,977	110,62
100	42	80	121	121	1	5.050	1,974	111,04
100	137	61	121	121	4	5.050	1,922	108,78
100	2026	75	121	121	0	5.050	1,936	110,64
200	42	80	118	121	1	10.050	4,239	108,32
200	137	61	118	121	4	10.050	3,786	111,00
200	2026	75	121	121	0	10.050	4,011	111,70

Pengaruh Banyak Generasi. Seluruh konfigurasi generasi menemukan best objective 121 pada ketiga seed. Best generation selalu berada pada generasi 0, 1, atau 4. Dengan demikian, penambahan generasi dari 50 menjadi 200 tidak meningkatkan best objective pada dataset ini, sedangkan median evaluasi meningkat dari 2.550 menjadi 10.050 dan median durasi dari 0,961 menjadi 4,011 detik. Penurunan final objective pada sejumlah run 50 dan 200 generasi konsisten dengan tidak digunakannya elitisme. Best state tetap tersimpan walaupun populasi terakhir memburuk.

Tabel 8: Sweep ukuran populasi GA dengan jumlah generasi tetap 100.

P	Seed	$f(s_0)$	$f(s_f)$	$f(s^*)$	Gen. terbaik	Evaluasi	Durasi (s)	Rerata pop. akhir
20	42	80	117	121	36	2.020	0,612	111,40
20	137	61	121	121	38	2.020	0,657	108,20
20	2026	75	114	120	73	2.020	0,497	104,80
50	42	80	121	121	1	5.050	1,855	111,04
50	137	61	121	121	4	5.050	1,839	108,78
50	2026	75	121	121	0	5.050	1,937	110,64
100	42	80	121	121	2	10.100	4,488	110,15
100	137	61	121	121	6	10.100	4,374	110,64
100	2026	75	121	121	0	10.100	4,345	107,83

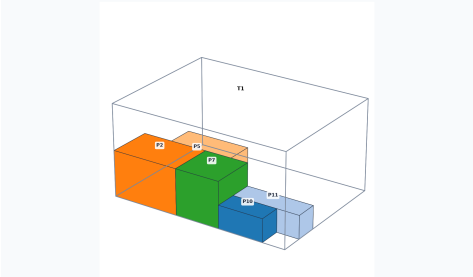
Pengaruh Ukuran Populasi. Populasi 20 gagal mencapai 121 pada seed 2026 dan berhenti pada best objective 120. Populasi 50 dan 100 mencapai 121 pada seluruh seed. Peningkatan populasi dari 50 menjadi 100 tidak meningkatkan best objective, tetapi menggandakan median evaluasi dari 5.050 menjadi 10.100 dan menaikkan median durasi dari 1,855 menjadi 4,374 detik. Untuk dataset dan tiga seed ini, populasi 50 dengan 100 generasi memberikan hasil yang sama dengan populasi 100 pada biaya lebih rendah. Kesimpulan ini tidak digeneralisasi ke dataset lain.



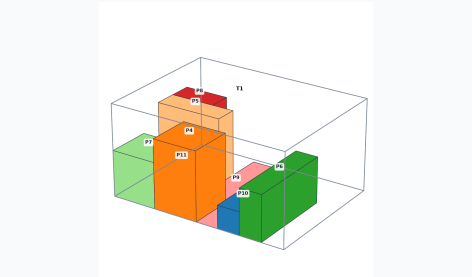
Sweep generasi: objective median Sweep populasi: objective median

Gambar 17: Perbandingan best objective pada dua sweep parameter GA.

Pasangan initial state dan best state untuk seluruh konfigurasi sweep GA ditampilkan berikut ini.

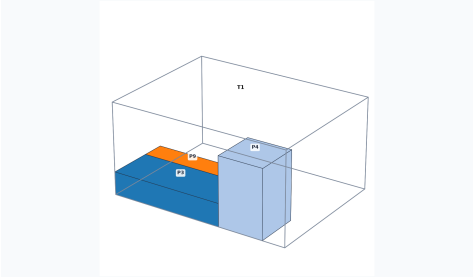


Initial state

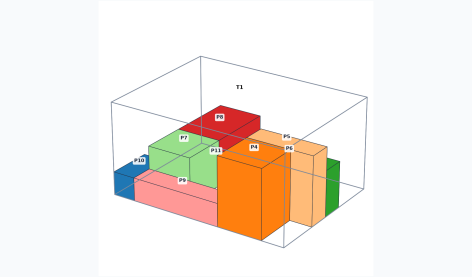


Best state

Gambar 18: GA P=50, G=50, seed 42, $f(s_0) = 80$ dan $f(s^*) = 121$.

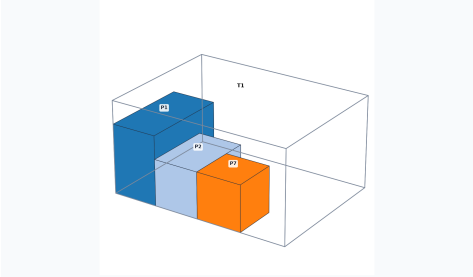


Initial state

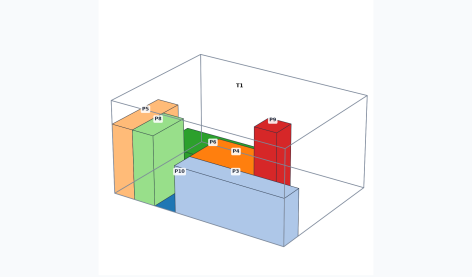


Best state

Gambar 19: GA P=50, G=50, seed 137, $f(s_0) = 61$ dan $f(s^*) = 121$.

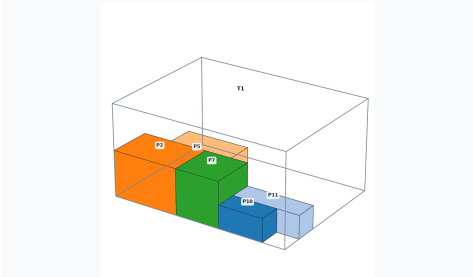


Initial state

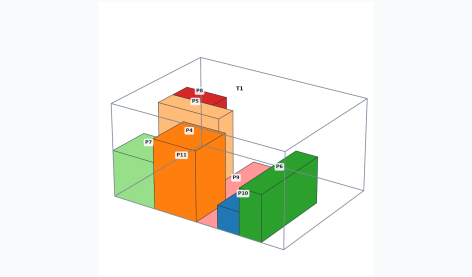


Best state

Gambar 20: GA P=50, G=50, seed 2026, $f(s_0) = 75$ dan $f(s^*) = 121$.

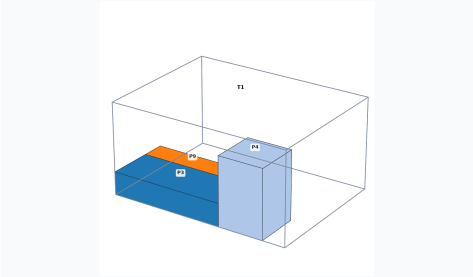


Initial state

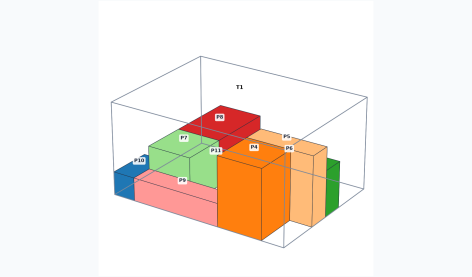


Best state

Gambar 21: GA P=50, G=100, seed 42, $f(s_0) = 80$ dan $f(s^*) = 121$.

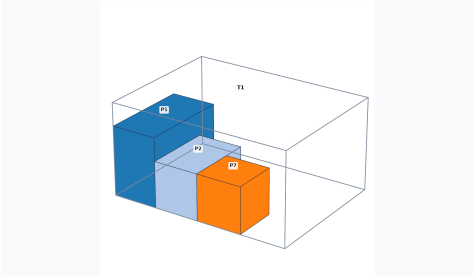


Initial state

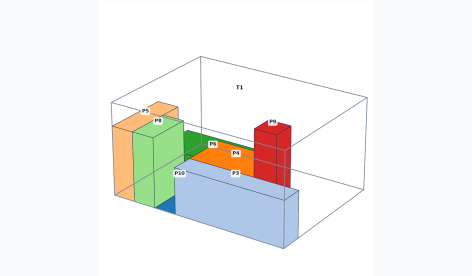


Best state

Gambar 22: GA P=50, G=100, seed 137, $f(s_0) = 61$ dan $f(s^*) = 121$.

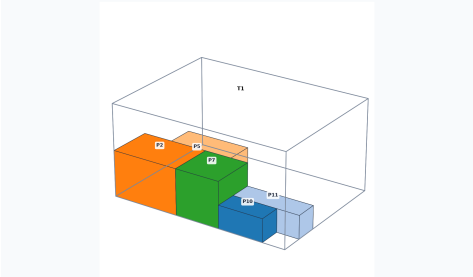


Initial state

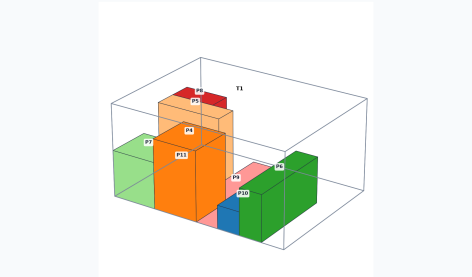


Best state

Gambar 23: GA P=50, G=100, seed 2026, $f(s_0) = 75$ dan $f(s^*) = 121$.

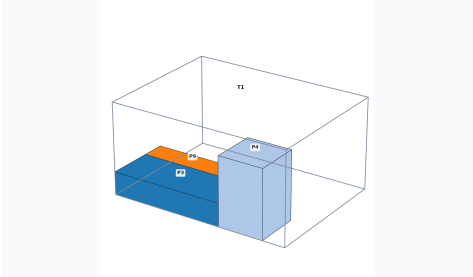


Initial state

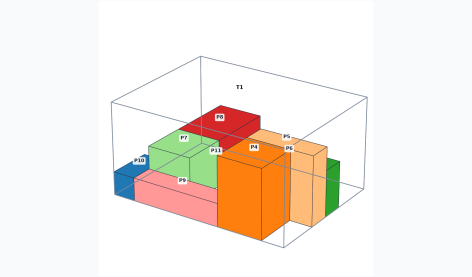


Best state

Gambar 24: GA P=50, G=200, seed 42, $f(s_0) = 80$ dan $f(s^*) = 121$.

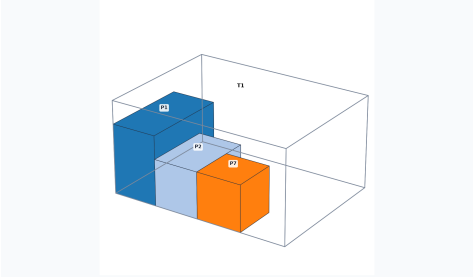


Initial state

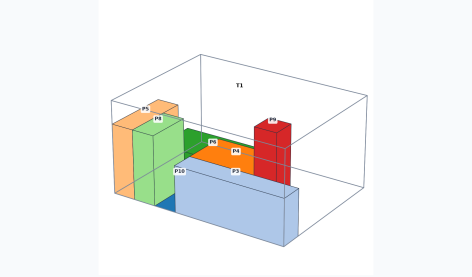


Best state

Gambar 25: GA P=50, G=200, seed 137, $f(s_0) = 61$ dan $f(s^*) = 121$.

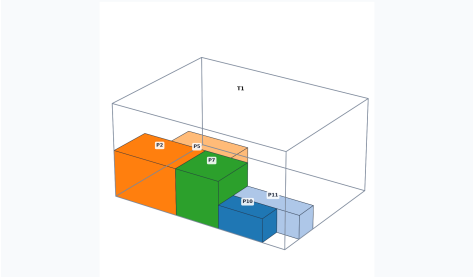


Initial state

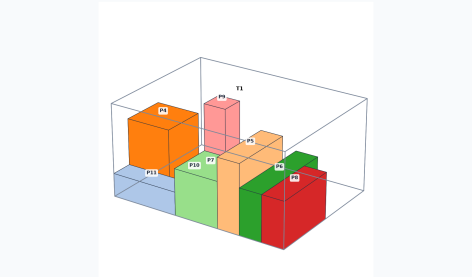


Best state

Gambar 26: GA P=50, G=200, seed 2026, $f(s_0) = 75$ dan $f(s^*) = 121$.

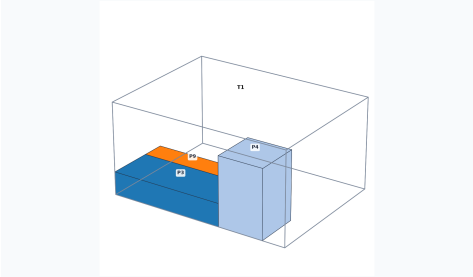


Initial state

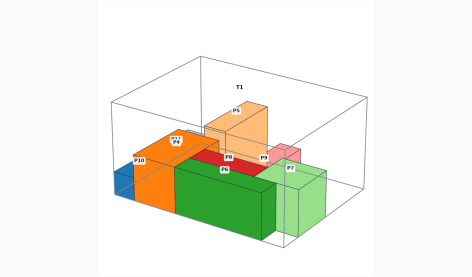


Best state

Gambar 27: GA P=20, G=100, seed 42, $f(s_0) = 80$ dan $f(s^*) = 121$.

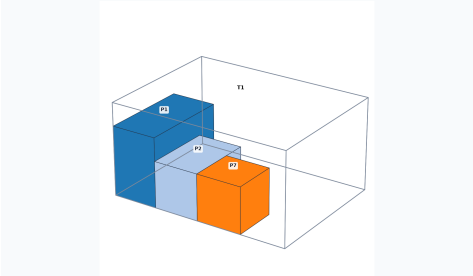


Initial state

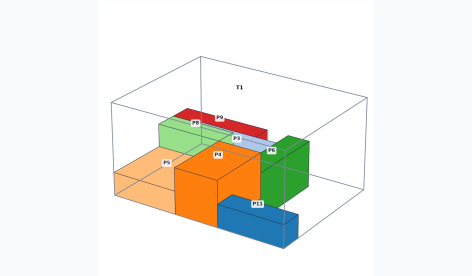


Best state

Gambar 28: GA P=20, G=100, seed 137, $f(s_0) = 61$ dan $f(s^*) = 121$.

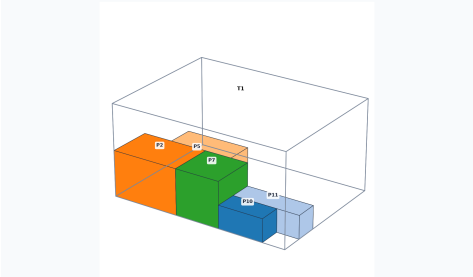


Initial state

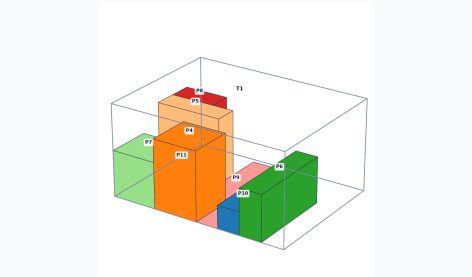


Best state

Gambar 29: GA P=20, G=100, seed 2026, $f(s_0) = 75$ dan $f(s^*) = 120$.

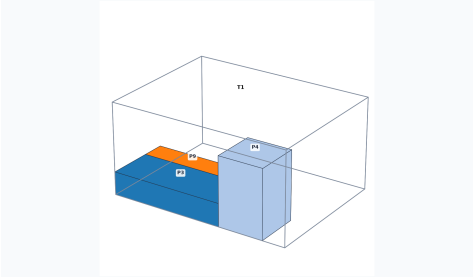


Initial state

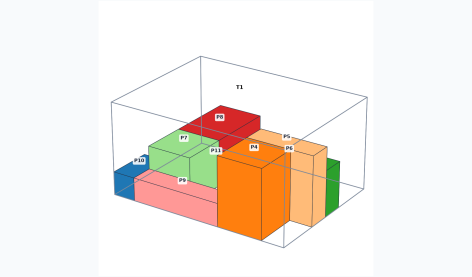


Best state

Gambar 30: GA P=50, G=100, seed 42, $f(s_0) = 80$ dan $f(s^*) = 121$.

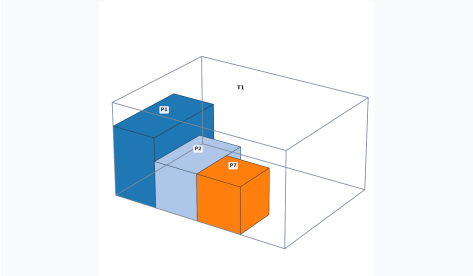


Initial state

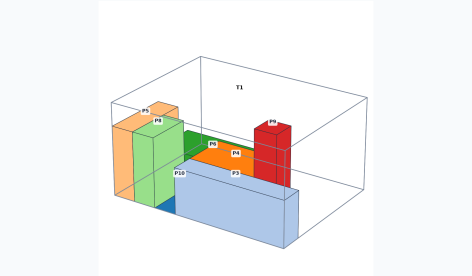


Best state

Gambar 31: GA P=50, G=100, seed 137, $f(s_0) = 61$ dan $f(s^*) = 121$.

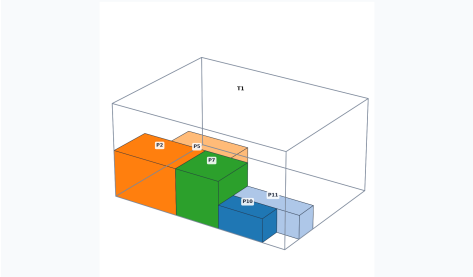


Initial state

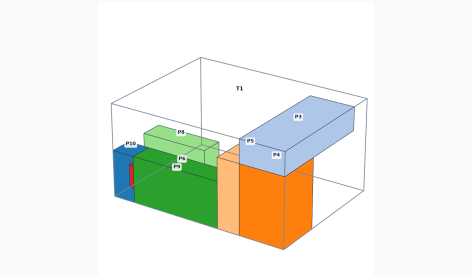


Best state

Gambar 32: GA P=50, G=100, seed 2026, $f(s_0) = 75$ dan $f(s^*) = 121$.

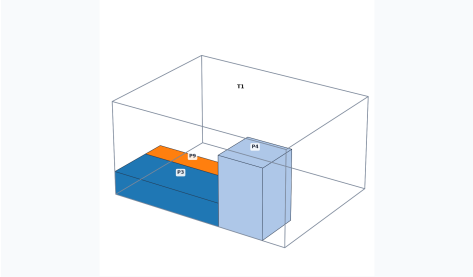


Initial state

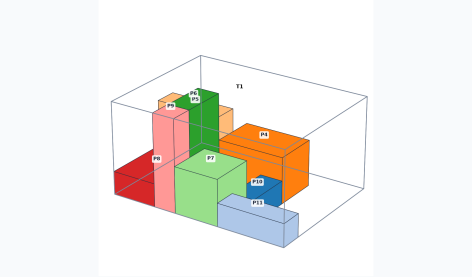


Best state

Gambar 33: GA P=100, G=100, seed 42, $f(s_0) = 80$ dan $f(s^*) = 121$.

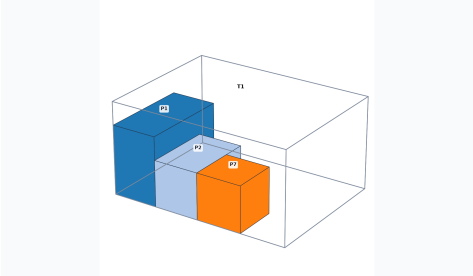


Initial state

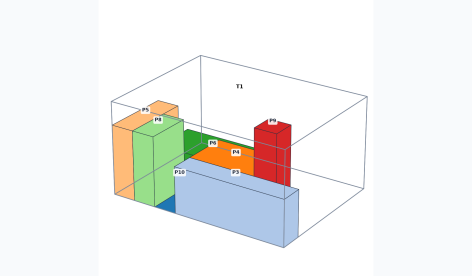


Best state

Gambar 34: GA P=100, G=100, seed 137, $f(s_0) = 61$ dan $f(s^*) = 121$.



Initial state



Best state

Gambar 35: GA P=100, G=100, seed 2026, $f(s_0) = 75$ dan $f(s^*) = 121$.

4.2.4 Analisis Perbandingan Algoritma

Tabel 9 merangkum kualitas solusi, jumlah evaluasi, dan durasi setiap algoritma pada dataset single-truck. Nilai median digunakan karena setiap konfigurasi dijalankan dengan tiga seed. GA dibandingkan pada konfigurasi $P = 50$ dan $G = 100$, sedangkan angka evaluasi yang ditampilkan merupakan jumlah aktual sampai kriteria berhenti masing-masing metode tercapai.

Tabel 9: Perbandingan utama pada dataset single-truck.

Algoritma	Rentang $f(s^*)$	Median	% UB 145	Median eval.	Median durasi (s)	Berhenti
Steepest-ascent HC	104–113	106	73,10	101	0,0086	converged
HC sideways	109–117	109	75,17	6.087	0,3784	sideways limit
Stochastic HC	104–113	106	73,10	101	0,0078	converged
Random-restart HC	116–121	121	83,45	885	0,1125	restart limit
Simulated annealing	121–121	121	83,45	50.000	13,3520	budget exhausted
GA ($P = 50, G = 100$)	121–121	121	83,45	5.050	1,9357	generation limit

Hasil pengujian menunjukkan bahwa random-restart HC, SA, dan GA memberikan kualitas terbaik yang sama dengan median objective 121. SA dan GA mencapai nilai tersebut pada seluruh seed, sedangkan random-restart HC mencapainya pada dua dari tiga seed. Random-restart HC menjadi metode paling hemat di antara ketiganya dengan median 885 evaluasi dan durasi 0,1125 detik, tetapi hasilnya lebih dipengaruhi seed. GA menawarkan hasil yang konsisten dengan 5.050 evaluasi, jauh lebih sedikit daripada 50.000 evaluasi yang digunakan SA. Dengan demikian, random-restart HC paling efisien pada konfigurasi uji ini, sedangkan GA memberikan keseimbangan paling baik antara konsistensi hasil dan biaya pencarian. Nilai 121 tetap diperlakukan sebagai nilai terbaik yang teramati, bukan optimum global, karena exact solver tidak dijalankan pada kasus 12 paket dan upper bound 145 belum memperhitungkan seluruh constraint ruang.

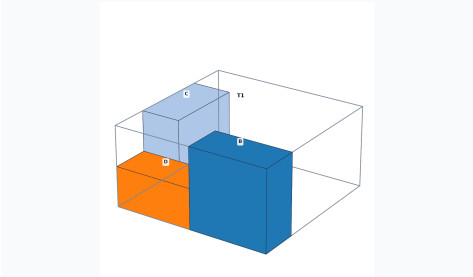
4.2.5 Pembanding Exact pada Dataset Tiny

Optimum exact untuk dataset tiny adalah 36, diperoleh dengan memuat paket B, C, dan D. Steepest-ascent, sideways, dan stochastic mencapai 36 pada dua dari tiga seed dan 35 pada satu seed. Random-restart, SA, dan GA mencapai 36 pada ketiga seed. Oleh karena itu, pada kasus kecil ini nilai terbaik random-restart, SA, dan GA dapat dinyatakan optimal secara global, sedangkan tiga varian HC lain memiliki tingkat keberhasilan 2/3 terhadap optimum exact.

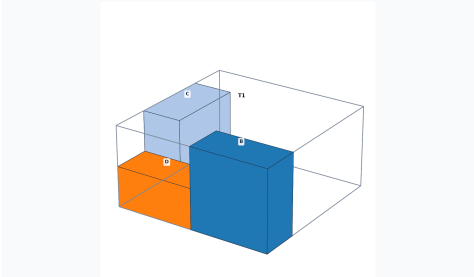
Tabel 10: Kualitas terhadap optimum exact 36 pada dataset tiny.

Algoritma	Minimum	Median	Maksimum	Run mencapai optimum
Steepest-ascent HC	35	36	36	2/3
HC sideways	35	36	36	2/3
Stochastic HC	35	36	36	2/3
Random-restart HC	36	36	36	3/3
Simulated annealing	36	36	36	3/3
Genetic algorithm	36	36	36	3/3

Pasangan initial state dan best state untuk seluruh eksperimen pada dataset tiny ditampilkan berikut ini.

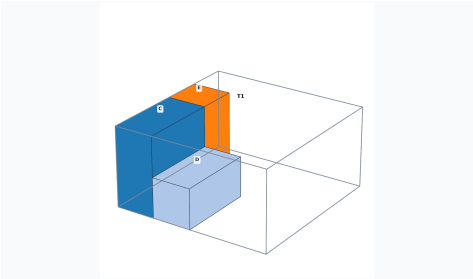


Initial state

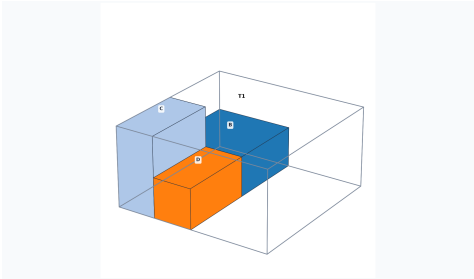


Best state

Gambar 36: Steepest-ascent HC, seed 42, $f(s_0) = 36$ dan $f(s^*) = 36$.

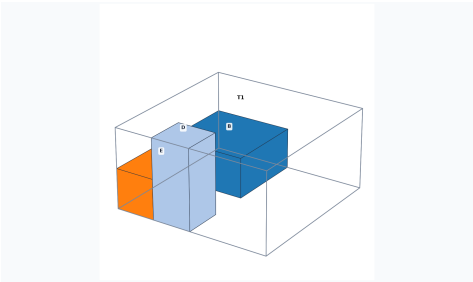


Initial state

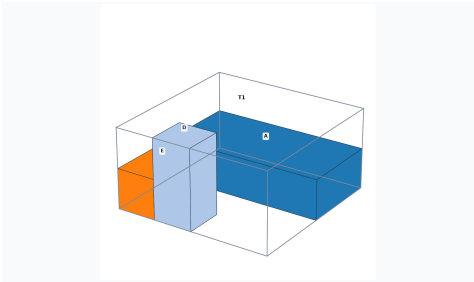


Best state

Gambar 37: Steepest-ascent HC, seed 137, $f(s_0) = 30$ dan $f(s^*) = 36$.

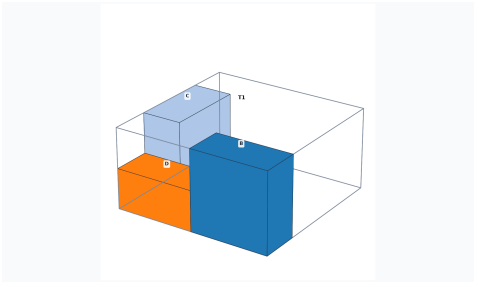


Initial state



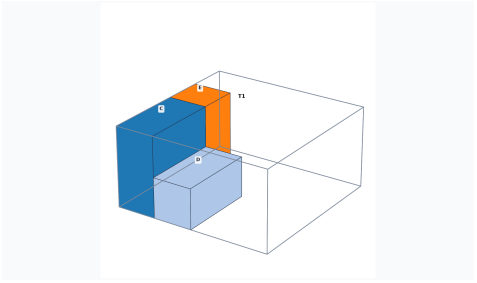
Best state

Gambar 38: Steepest-ascent HC, seed 2026, $f(s_0) = 31$ dan $f(s^*) = 35$.



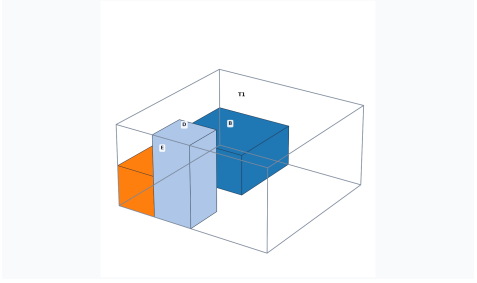
Initial state *Best state*

Gambar 39: HC sideways, seed 42, $f(s_0) = 36$ dan $f(s^*) = 36$.



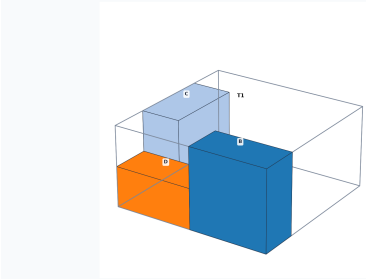
Initial state *Best state*

Gambar 40: HC sideways, seed 137, $f(s_0) = 30$ dan $f(s^*) = 36$.

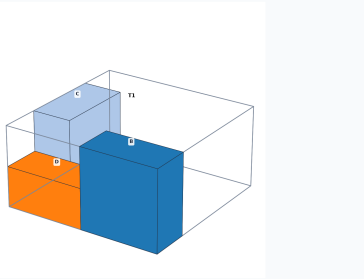


Initial state *Best state*

Gambar 41: HC sideways, seed 2026, $f(s_0) = 31$ dan $f(s^*) = 35$.

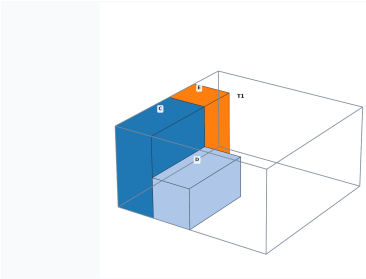


Initial state

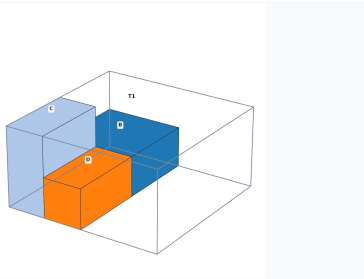


Best state

Gambar 42: Stochastic HC, seed 42, $f(s_0) = 36$ dan $f(s^*) = 36$.

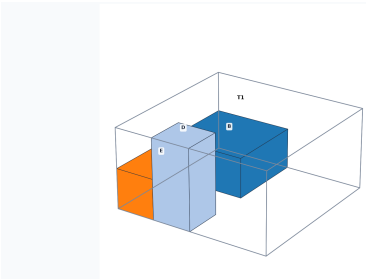


Initial state

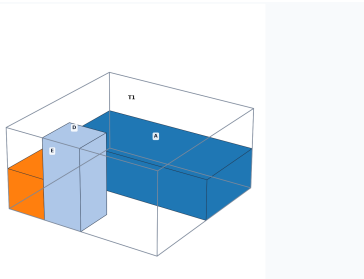


Best state

Gambar 43: Stochastic HC, seed 137, $f(s_0) = 30$ dan $f(s^*) = 36$.

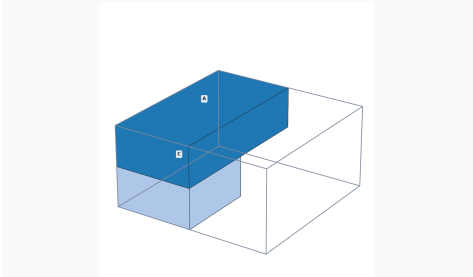


Initial state

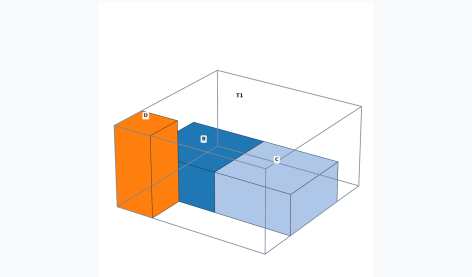


Best state

Gambar 44: Stochastic HC, seed 2026, $f(s_0) = 31$ dan $f(s^*) = 35$.

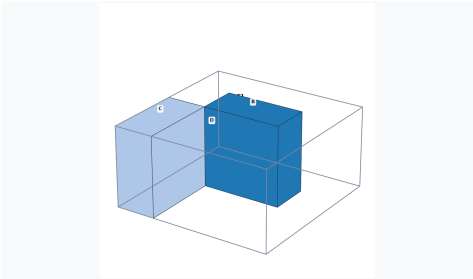


Initial state

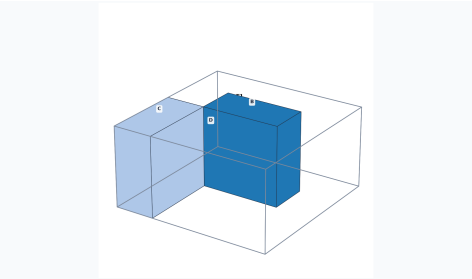


Best state

Gambar 45: Random-restart HC, seed 42, $f(s_0) = 31$ dan $f(s^*) = 36$.

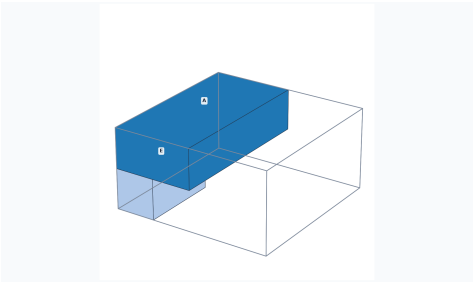


Initial state

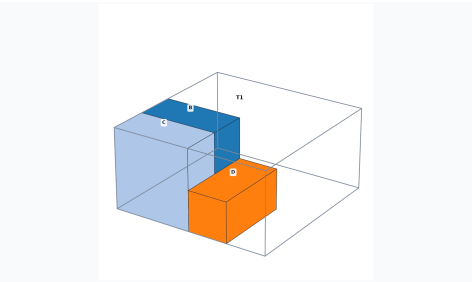


Best state

Gambar 46: Random-restart HC, seed 137, $f(s_0) = 36$ dan $f(s^*) = 36$.

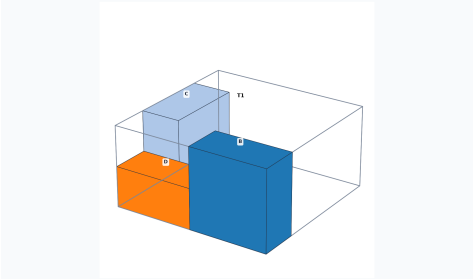


Initial state



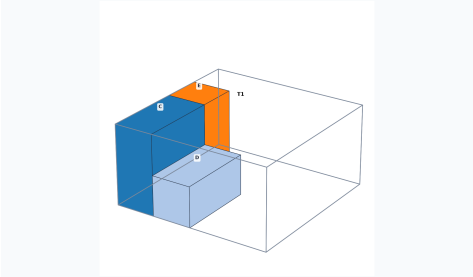
Best state

Gambar 47: Random-restart HC, seed 2026, $f(s_0) = 26$ dan $f(s^*) = 36$.



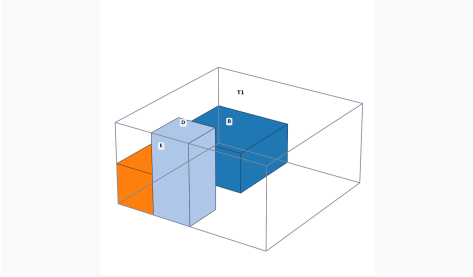
Initial state *Best state*

Gambar 48: Simulated annealing, seed 42, $f(s_0) = 36$ dan $f(s^*) = 36$.



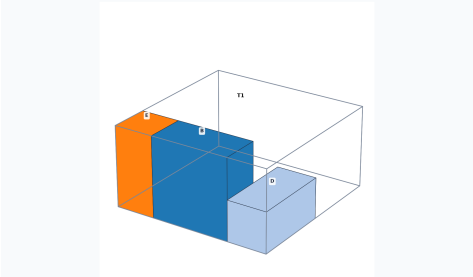
Initial state *Best state*

Gambar 49: Simulated annealing, seed 137, $f(s_0) = 30$ dan $f(s^*) = 36$.

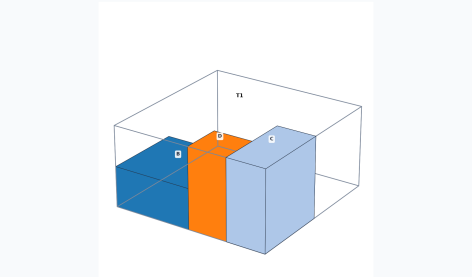


Initial state *Best state*

Gambar 50: Simulated annealing, seed 2026, $f(s_0) = 31$ dan $f(s^*) = 36$.

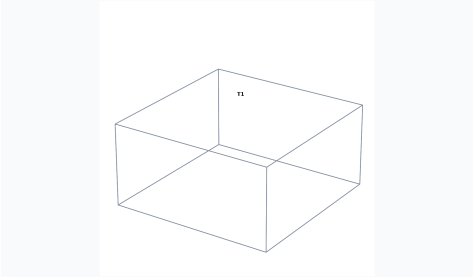


Initial state

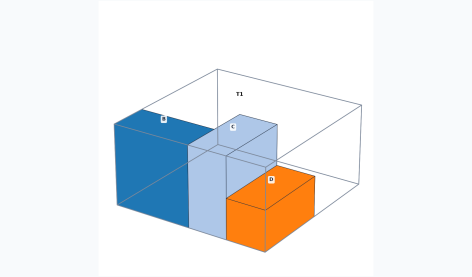


Best state

Gambar 51: Genetic algorithm, seed 42, $f(s_0) = 31$ dan $f(s^*) = 36$.

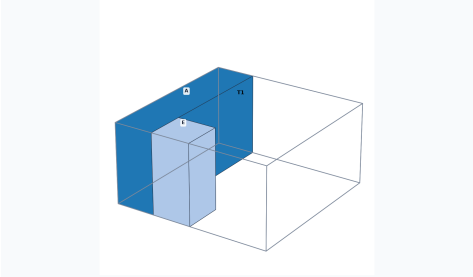


Initial state

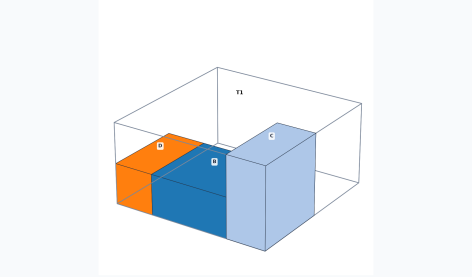


Best state

Gambar 52: Genetic algorithm, seed 137, $f(s_0) = 0$ dan $f(s^*) = 36$.



Initial state



Best state

Gambar 53: Genetic algorithm, seed 2026, $f(s_0) = 26$ dan $f(s^*) = 36$.

4.2.6 Bonus Multi-Truck

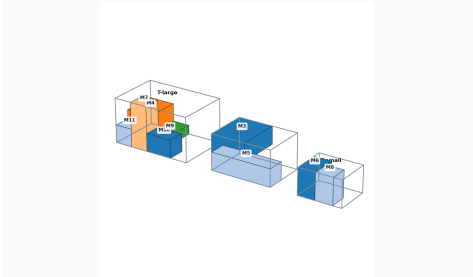
Dukungan multi-truck diterapkan dengan menyimpan identitas truk pada setiap penempatan paket. Validator memeriksa dimensi, kapasitas berat, overlap, support, dan aturan fragile secara terpisah untuk setiap truk. Operator neighbor dapat memindahkan paket ke truk lain, sedangkan decoder GA mencoba truk yang tersimpan pada gene sebelum menggunakan truk lain sebagai pilihan cadangan. Dengan mekanisme ini, penambahan truk tidak mengubah bentuk umum algoritma pencarian, tetapi memperluas alternatif penempatan yang dapat dievaluasi.

Seluruh run pada kasus tiga truk menghasilkan state valid. Nilai tertinggi yang teramati adalah 223, dicapai oleh random-restart HC dan GA dengan memuat paket M1, M2, M3, M7, M8, M9, M10, dan M11. GA menghasilkan median 222 dengan rentang yang sempit, sedangkan random-restart HC menghasilkan median 220 dengan biaya evaluasi yang lebih rendah. Hasil ini menunjukkan bahwa kedua metode mampu memanfaatkan kapasitas gabungan antartuk dengan baik. Rasio 89,56% terhadap upper bound 249 hanya digunakan sebagai acuan karena bound tersebut belum memperhitungkan seluruh constraint geometris.

Tabel 11: Hasil bonus multi-truck dengan objective total value.

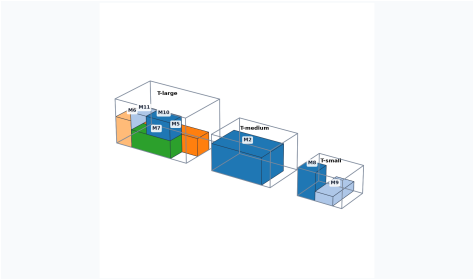
Algoritma	Min.	Median	Maks.	Median eval.	Median durasi (s)
Steepest-ascent HC	184	186	208	50	0,0073
HC sideways	200	208	212	3.939	0,5522
Stochastic HC	184	186	208	50	0,0070
Random-restart HC	218	220	223	766	0,1017
Simulated annealing	208	208	213	50.000	11,1093
Genetic algorithm	222	222	223	5.050	1,4429

Pasangan initial state dan best state untuk seluruh eksperimen multi-truck ditampilkan berikut ini.



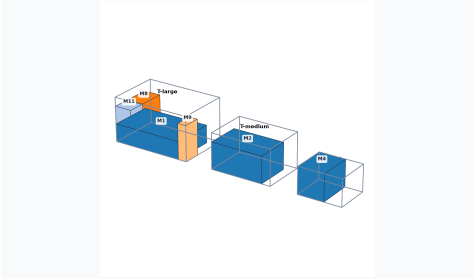
Initial state *Best state*

Gambar 54: Steepest-ascent HC, seed 42, $f(s_0) = 208$ dan $f(s^*) = 208$.



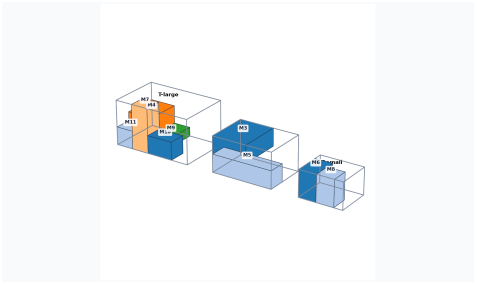
Initial state *Best state*

Gambar 55: Steepest-ascent HC, seed 137, $f(s_0) = 186$ dan $f(s^*) = 186$.

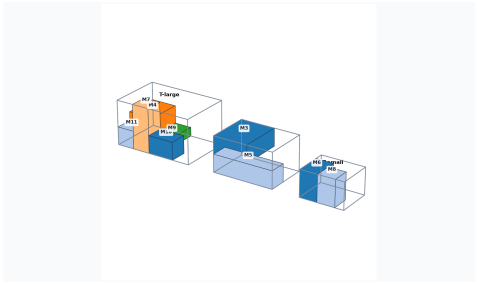


Initial state *Best state*

Gambar 56: Steepest-ascent HC, seed 2026, $f(s_0) = 180$ dan $f(s^*) = 184$.

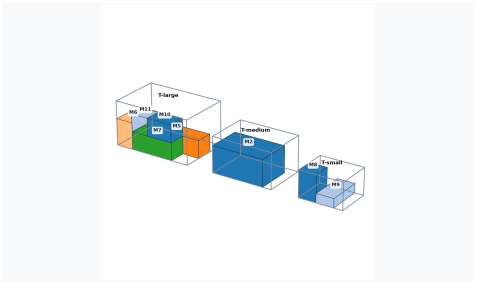


Initial state

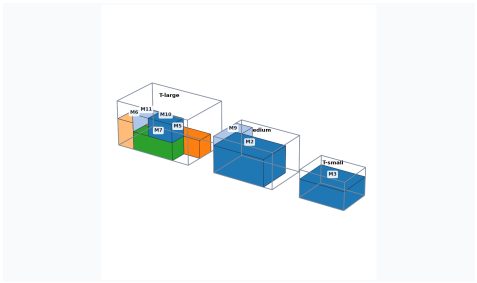


Best state

Gambar 57: HC sideways, seed 42, $f(s_0) = 208$ dan $f(s^*) = 208$.

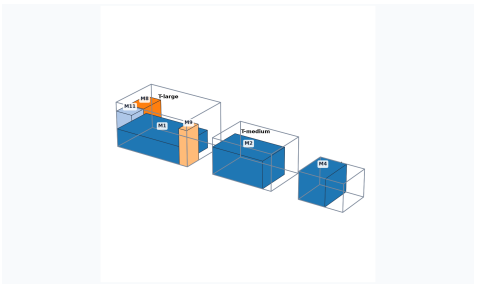


Initial state

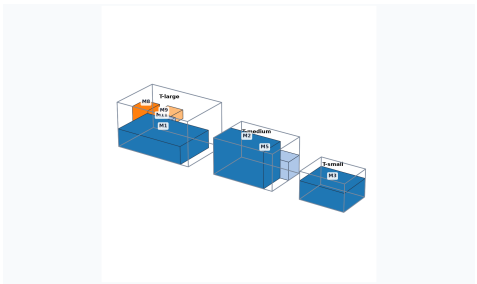


Best state

Gambar 58: HC sideways, seed 137, $f(s_0) = 186$ dan $f(s^*) = 200$.

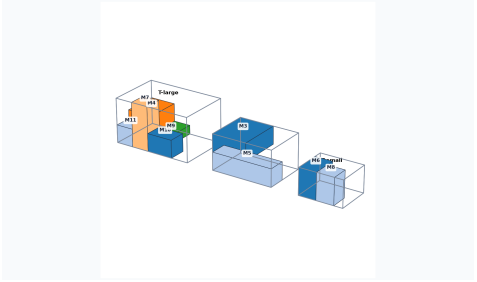


Initial state



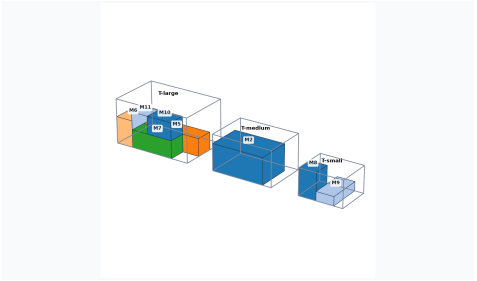
Best state

Gambar 59: HC sideways, seed 2026, $f(s_0) = 180$ dan $f(s^*) = 212$.



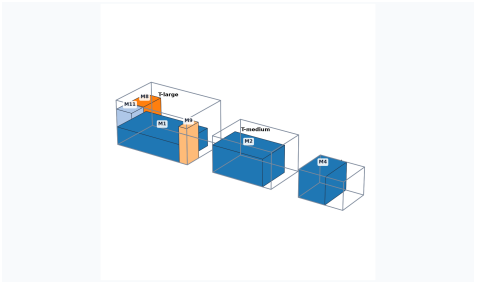
Initial state *Best state*

Gambar 60: Stochastic HC, seed 42, $f(s_0) = 208$ dan $f(s^*) = 208$.



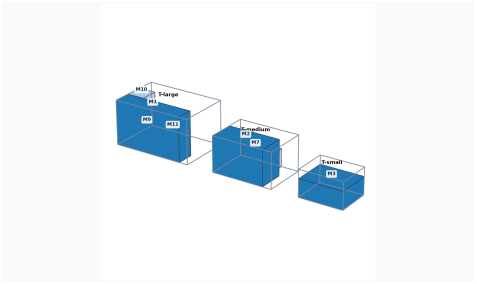
Initial state *Best state*

Gambar 61: Stochastic HC, seed 137, $f(s_0) = 186$ dan $f(s^*) = 186$.

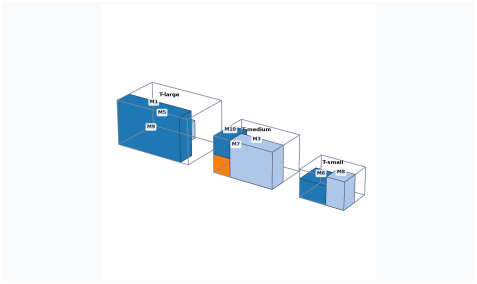


Initial state *Best state*

Gambar 62: Stochastic HC, seed 2026, $f(s_0) = 180$ dan $f(s^*) = 184$.

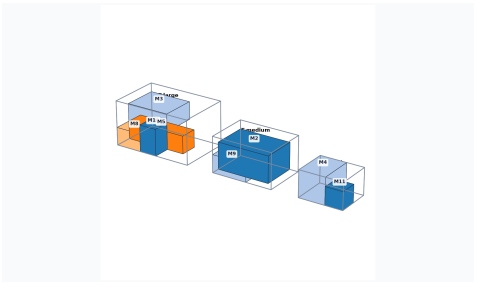


Initial state

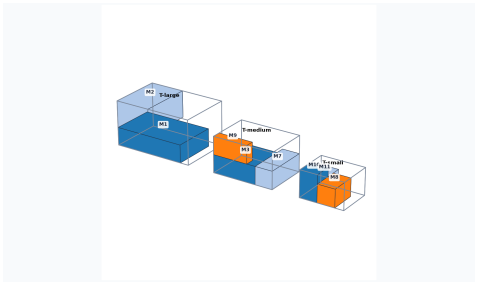


Best state

Gambar 63: Random-restart HC, seed 42, $f(s_0) = 203$ dan $f(s^*) = 218$.

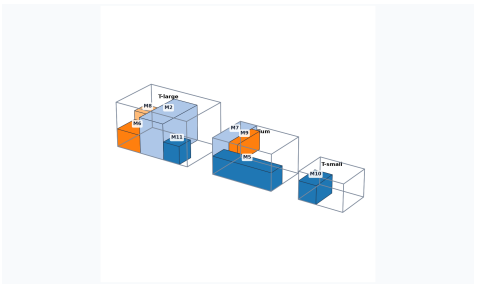


Initial state

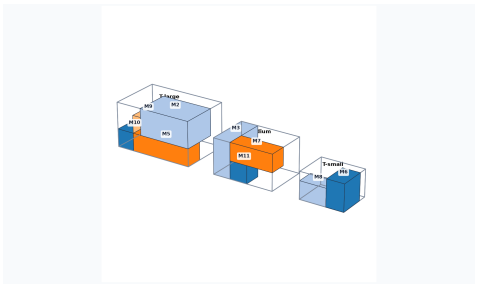


Best state

Gambar 64: Random-restart HC, seed 137, $f(s_0) = 204$ dan $f(s^*) = 223$.

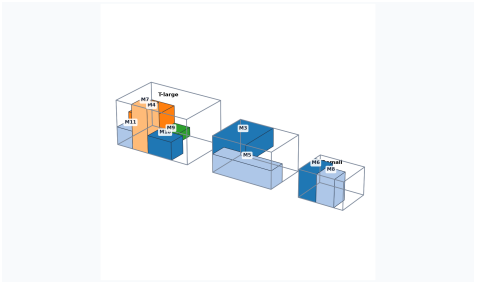


Initial state

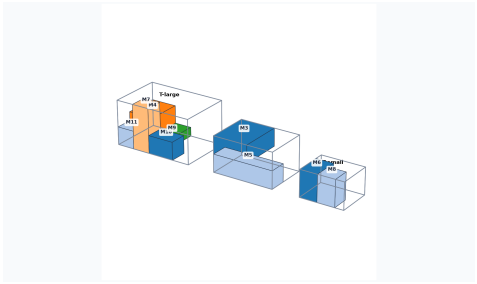


Best state

Gambar 65: Random-restart HC, seed 2026, $f(s_0) = 186$ dan $f(s^*) = 220$.

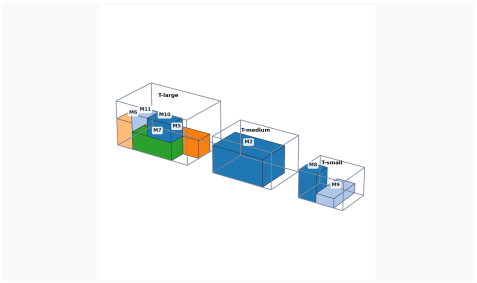


Initial state

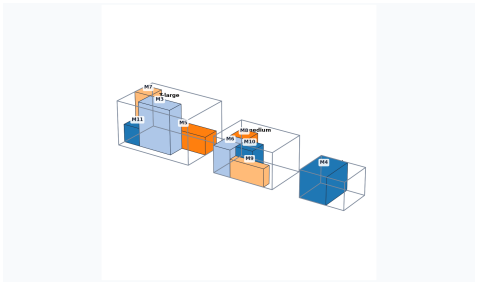


Best state

Gambar 66: Simulated annealing, seed 42, $f(s_0) = 208$ dan $f(s^*) = 208$.

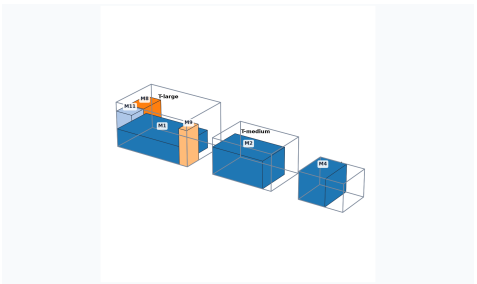


Initial state

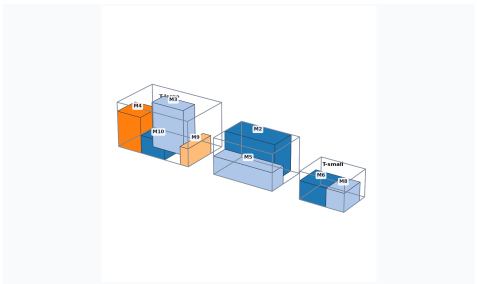


Best state

Gambar 67: Simulated annealing, seed 137, $f(s_0) = 186$ dan $f(s^*) = 208$.

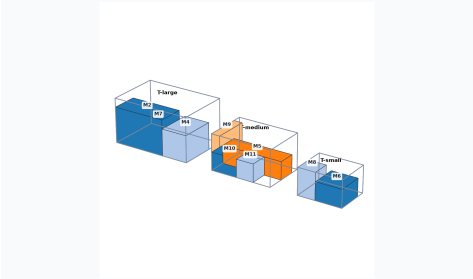


Initial state

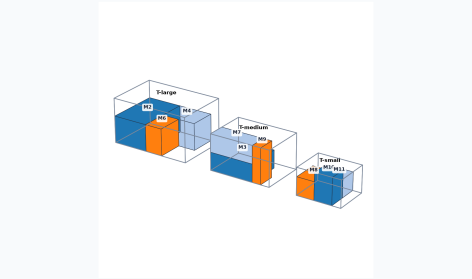


Best state

Gambar 68: Simulated annealing, seed 2026, $f(s_0) = 180$ dan $f(s^*) = 213$.

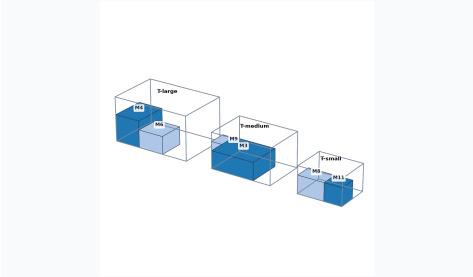


Initial state

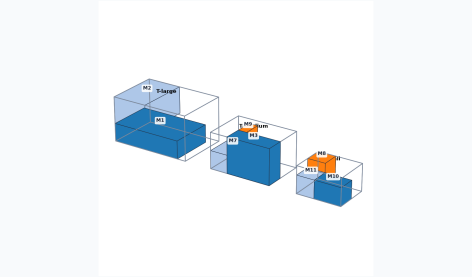


Best state

Gambar 69: Genetic algorithm, seed 42, $f(s_0) = 216$ dan $f(s^*) = 222$.

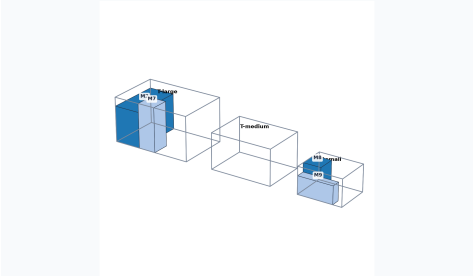


Initial state

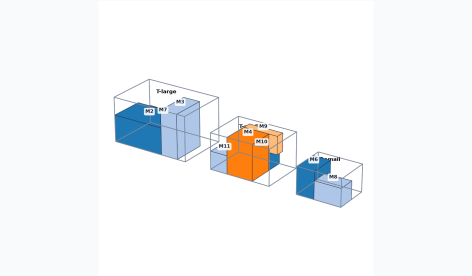


Best state

Gambar 70: Genetic algorithm, seed 137, $f(s_0) = 141$ dan $f(s^*) = 223$.



Initial state



Best state

Gambar 71: Genetic algorithm, seed 2026, $f(s_0) = 102$ dan $f(s^*) = 222$.

4.2.7 Bonus Objective Berbasis ETA

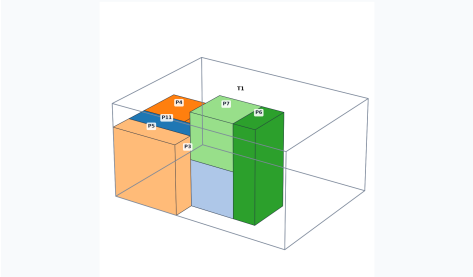
Objective bonus $v/(\text{ETA} + 1)$ mengubah dasar penilaian dari total value menjadi nilai yang mempertimbangkan urgensi pengiriman. Paket dengan value besar tetap menguntungkan, tetapi kontribusinya berkurang ketika ETA semakin tinggi. Implementasi menggunakan fungsi evaluasi dan validator yang sama dengan objective utama, sehingga perbedaannya hanya terletak pada skor paket yang dijumlahkan dan bukan pada aturan kelayakan penempatan.

Pada dataset single-truck, nilai tertinggi yang teramati adalah 79,833 dengan paket P2, P3, P4, P6, P8, dan P10. Random-restart HC dan GA mencapai nilai ini pada seluruh seed. SA mencapainya pada dua seed, sedangkan HC sideways mencapainya pada satu seed. Konsistensi random-restart HC dan GA menunjukkan bahwa kedua metode mampu menyesuaikan pencarian ketika prioritas solusi tidak lagi ditentukan oleh total value semata. Karena tidak tersedia pembanding exact untuk kasus 12 paket, 79,833 dinyatakan sebagai nilai terbaik yang teramati dan bukan sebagai optimum global.

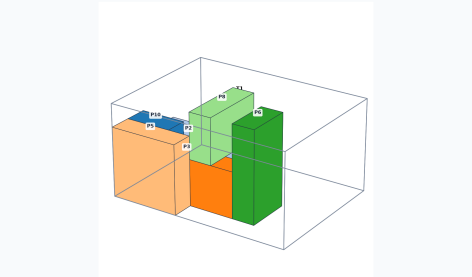
Tabel 12: Hasil bonus objective $v/(\text{ETA} + 1)$.

Algoritma	Minimum	Median	Maksimum	Median eval.	Median durasi (s)
Steepest-ascent HC	71,833	79,167	79,167	308	0,0221
HC sideways	79,167	79,167	79,833	5.899	0,7269
Stochastic HC	67,333	79,167	79,167	292	0,0200
Random-restart HC	79,833	79,833	79,833	1.517	0,1841
Simulated annealing	79,167	79,833	79,833	50.000	11,9023
Genetic algorithm	79,833	79,833	79,833	5.050	1,9438

Pasangan initial state dan best state untuk seluruh eksperimen dengan objective berbasis ETA ditampilkan berikut ini.

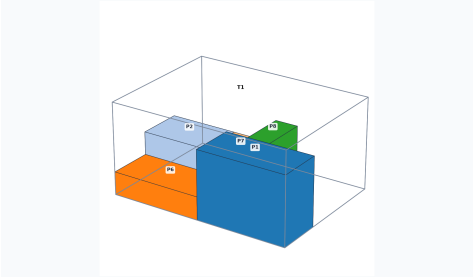


Initial state

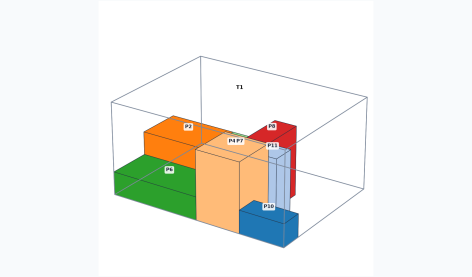


Best state

Gambar 72: Steepest-ascent HC, seed 42, $f(s_0) = 50$ dan $f(s^*) = 71,833$.

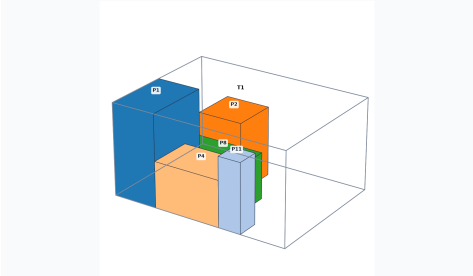


Initial state

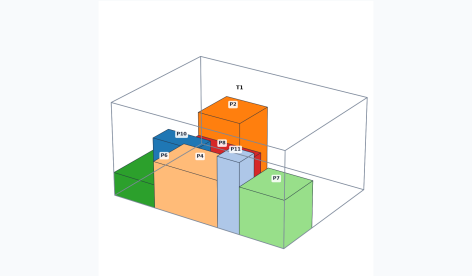


Best state

Gambar 73: Steepest-ascent HC, seed 137, $f(s_0) = 64,167$ dan $f(s^*) = 79,167$.

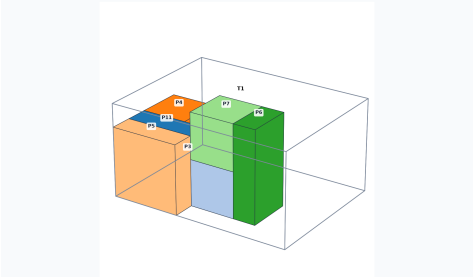


Initial state

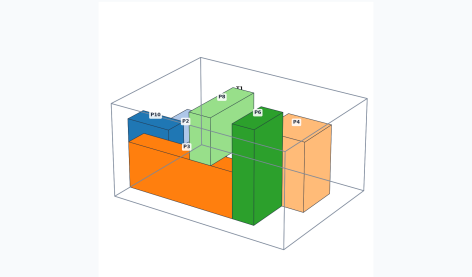


Best state

Gambar 74: Steepest-ascent HC, seed 2026, $f(s_0) = 54,500$ dan $f(s^*) = 79,167$.

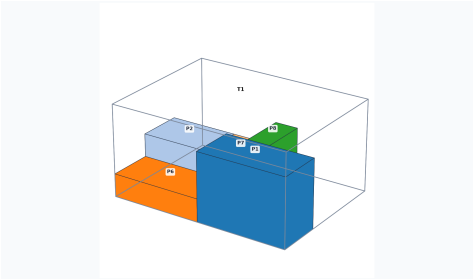


Initial state

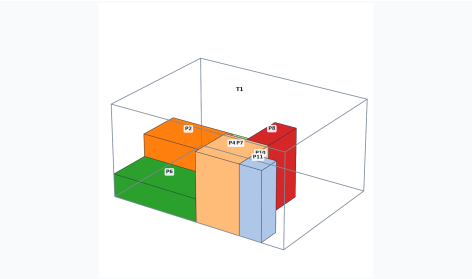


Best state

Gambar 75: HC sideways, seed 42, $f(s_0) = 50$ dan $f(s^*) = 79,833$.

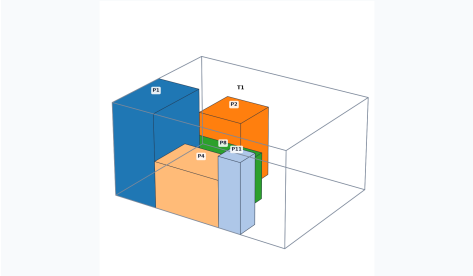


Initial state

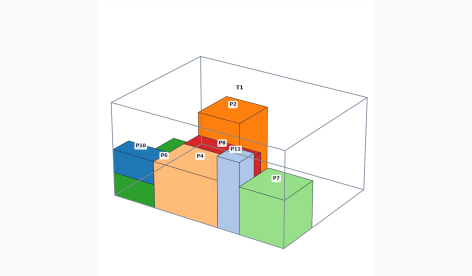


Best state

Gambar 76: HC sideways, seed 137, $f(s_0) = 64,167$ dan $f(s^*) = 79,167$.

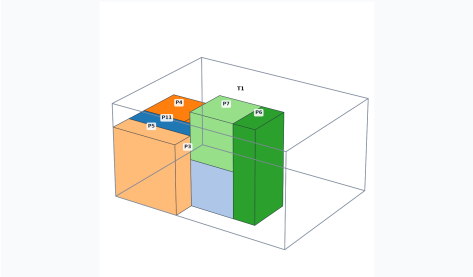


Initial state

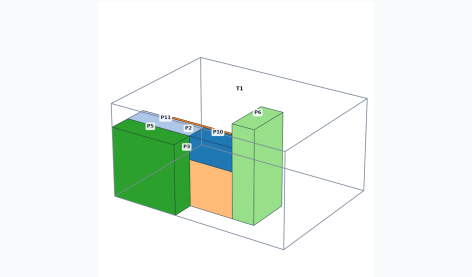


Best state

Gambar 77: HC sideways, seed 2026, $f(s_0) = 54,500$ dan $f(s^*) = 79,167$.

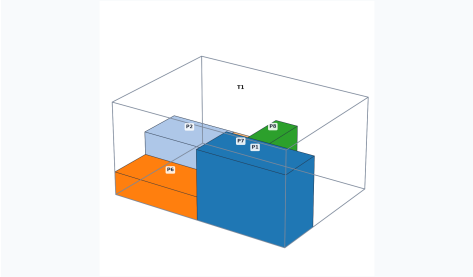


Initial state

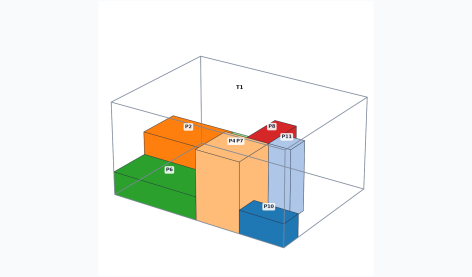


Best state

Gambar 78: Stochastic HC, seed 42, $f(s_0) = 50$ dan $f(s^*) = 67,333$.

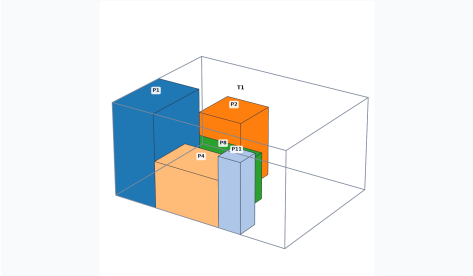


Initial state

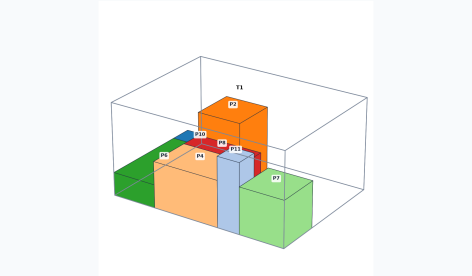


Best state

Gambar 79: Stochastic HC, seed 137, $f(s_0) = 64,167$ dan $f(s^*) = 79,167$.

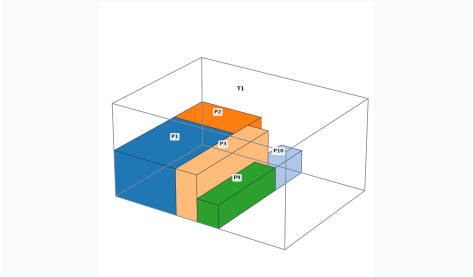


Initial state

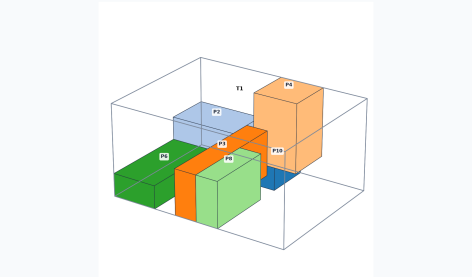


Best state

Gambar 80: Stochastic HC, seed 2026, $f(s_0) = 54,500$ dan $f(s^*) = 79,167$.

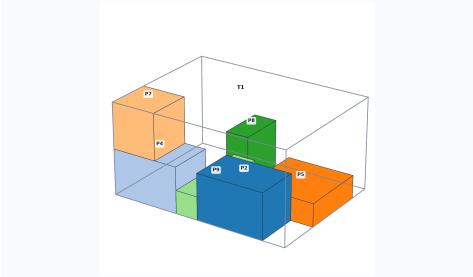


Initial state

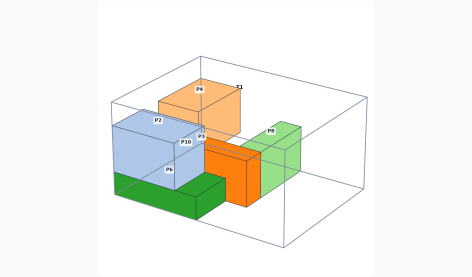


Best state

Gambar 81: Random-restart HC, seed 42, $f(s_0) = 53,333$ dan $f(s^*) = 79,833$.

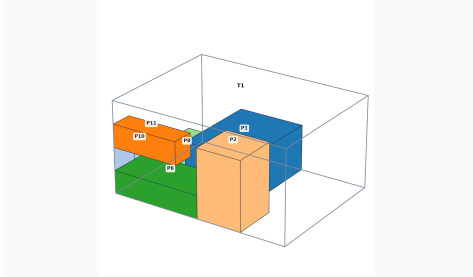


Initial state

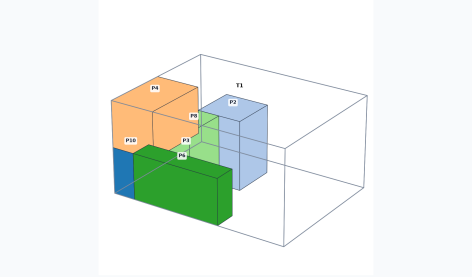


Best state

Gambar 82: Random-restart HC, seed 137, $f(s_0) = 56,167$ dan $f(s^*) = 79,833$.

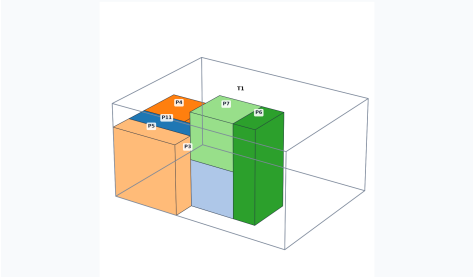


Initial state

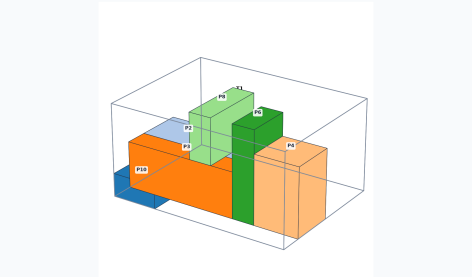


Best state

Gambar 83: Random-restart HC, seed 2026, $f(s_0) = 69,500$ dan $f(s^*) = 79,833$.

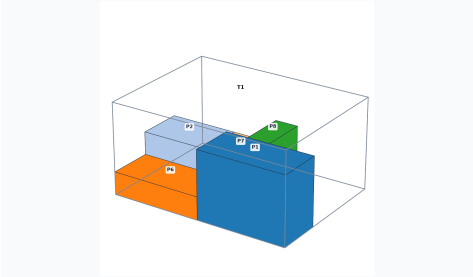


Initial state

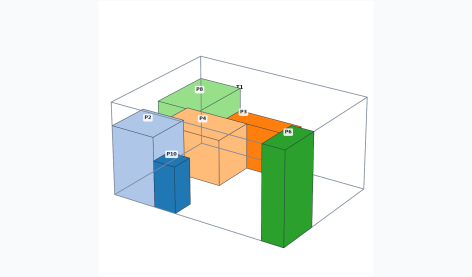


Best state

Gambar 84: Simulated annealing, seed 42, $f(s_0) = 50$ dan $f(s^*) = 79,833$.

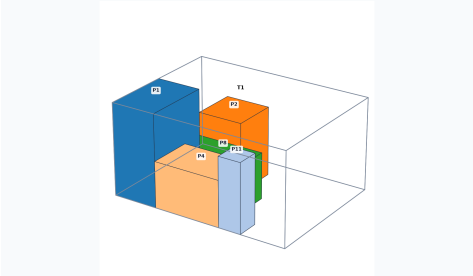


Initial state

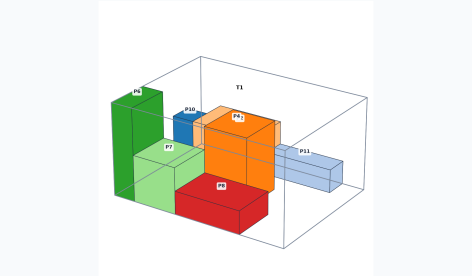


Best state

Gambar 85: Simulated annealing, seed 137, $f(s_0) = 64,167$ dan $f(s^*) = 79,833$.

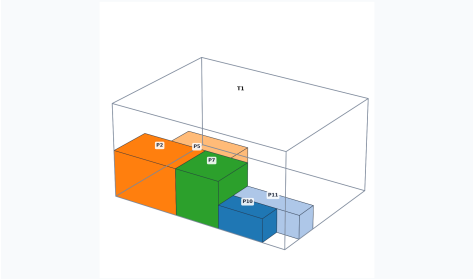


Initial state

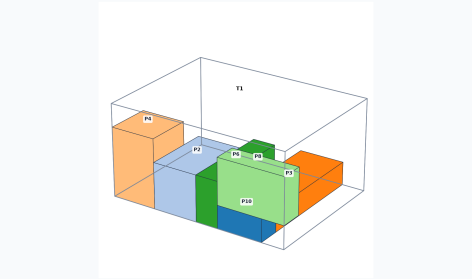


Best state

Gambar 86: Simulated annealing, seed 2026, $f(s_0) = 54,500$ dan $f(s^*) = 79,167$.

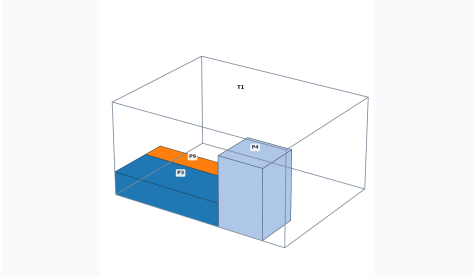


Initial state

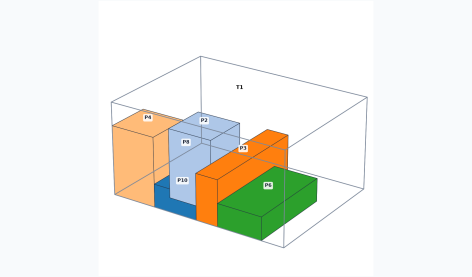


Best state

Gambar 87: Genetic algorithm, seed 42, $f(s_0) = 46,667$ dan $f(s^*) = 79,833$.

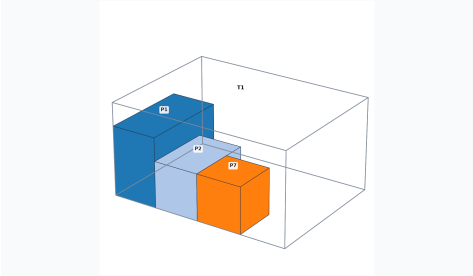


Initial state

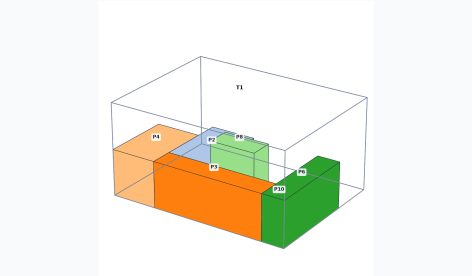


Best state

Gambar 88: Genetic algorithm, seed 137, $f(s_0) = 22,333$ dan $f(s^*) = 79,833$.

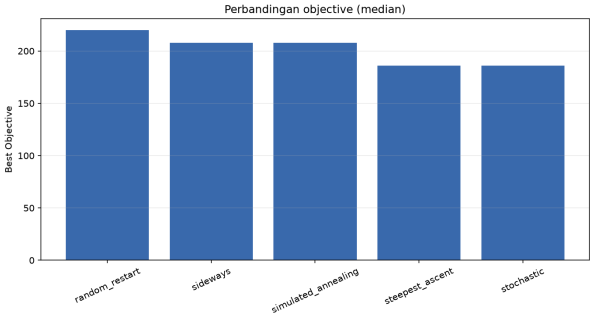


Initial state

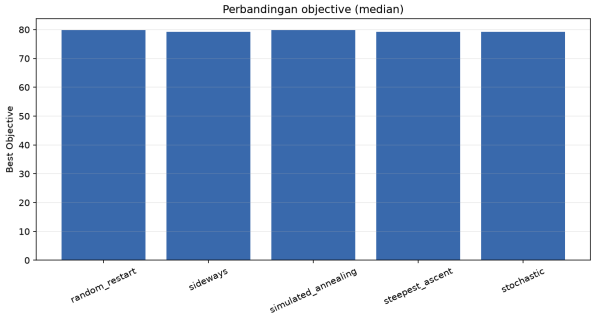


Best state

Gambar 89: Genetic algorithm, seed 2026, $f(s_0) = 39,667$ dan $f(s^*) = 79,833$.



Multi-truck, total value



Single-truck, $v/(\text{ETA} + 1)$

Gambar 90: Median objective lima algoritma non-GA pada dua evaluasi bonus.

5 Kesimpulan

Program berhasil mengimplementasikan empat varian Hill-Climbing, Simulated Annealing, dan Genetic Algorithm untuk optimasi pemuatan paket dengan mempertimbangkan posisi, orientasi, berat, support, overlap, dan aturan paket fragile. Pada dataset tiny, random-restart HC, SA, dan GA mencapai optimum exact 36 pada seluruh seed. Steepest-ascent HC, HC sideways, dan stochastic HC mencapai nilai tersebut pada dua dari tiga seed.

Pada dataset utama single-truck, random-restart HC, SA, dan GA menghasilkan median terbaik yang sama, yaitu 121. Random-restart HC menggunakan evaluasi dan durasi paling rendah, tetapi hasilnya lebih bervariasi. SA selalu mencapai 121 dengan biaya tertinggi, sedangkan GA selalu mencapai 121 dengan jumlah evaluasi sekitar sepersepuluh SA. Berdasarkan hasil tersebut, GA memberikan keseimbangan paling baik antara konsistensi dan biaya pencarian, sementara random-restart HC menjadi pilihan paling efisien apabila variasi antarseed masih dapat diterima. Sweep parameter GA juga menunjukkan bahwa populasi 50 dan 100 sama-sama stabil, sedangkan penambahan generasi di atas 50 tidak meningkatkan nilai terbaik pada dataset ini.

Fitur bonus dapat digunakan tanpa mengubah validitas solusi. Pada kasus multi-truck, nilai tertinggi yang teramati adalah 223 dan dicapai oleh random-restart HC serta GA. Pada objective berbasis ETA, nilai tertinggi yang teramati adalah 79,833 dan dicapai oleh random-restart HC serta GA pada seluruh seed. Temuan ini memperlihatkan bahwa representasi state, validator, dan mekanisme pencarian dapat digunakan kembali untuk variasi jumlah truk maupun fungsi penilaian. Nilai 121, 223, dan 79,833 tetap dinyatakan sebagai nilai terbaik yang teramati, bukan optimum global, karena pembanding exact hanya tersedia untuk dataset tiny.

6 Pembagian Tugas Anggota Kelompok

Tabel 13: Pembagian tugas anggota kelompok

NIM	Nama	Kontribusi
13524032	Juan Oloando Simanungkalit	Prototype UI, integrasi dan perbaikan bug, Genetic Algorithm, bonus multi-truck, dan laporan.
13524044	Narendra Dharma Wistara M.	UI, representasi state, pembentukan neighbor, fungsi objektif, Simulated Annealing, dan laporan.
13524056	Reinhard Alfonzo Hutabarat	Hill-Climbing, integrasi, fungsi objektif bonus, dan laporan.

Referensi

[1] Tim Asisten Laboratorium Inteligensi Artifisial 2023. *Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027*. Versi 1.4, 24 September 2026. Institut Teknologi Bandung.

[2] S. J. Russell dan P. Norvig. *Artificial Intelligence: A Modern Approach*, edisi ke-4. Pearson, 2021. Bab 4, Bagian 4.1.1, *Hill-climbing search*.

[3] Bean, J. C., "Genetic Algorithms and Random Keys for Sequencing and Optimization," *ORSA Journal on Computing*, vol. 6, no. 2, pp. 154–160, 1994, doi: 10.1287/ijoc.6.2.154.

[4] Materi kuliah IF3170. *Beyond Classical Search*. Materi Perkuliahan IF3170 Inteligensia Artifisial

A Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM :

1. Juan Oloando Simanungkalit/13524032
2. Narendra Dharma Wistara Marpaung/13524044
3. Reinhard Alfonzo Hutabarat/13524056

Fakultas/Sekolah : STEI-K
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Laporan Tugas Besar 1 IF3170 Inteligensi Artifisial

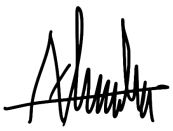
Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Kami menggunakan LLM untuk diskusi mengenai pemilihan pendekatan algoritma serta membantu merealisasikan GUI program.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Reinhard Alfonzo Hutabarat

13524056

B Data Eksperimen

Lampiran ini memuat data JSON yang digunakan pada ketiga kasus uji.

Listing 22: Kasus uji `tiny_exact.json`

```json
{
  "trucks": [
    {
      "id": "T1",
      "dimensions": {"width": 4, "length": 4, "height": 2},
      "max_capacity": 10
    }
  ],
  "packages": [
    {
      "id": "A",
      "dimensions": {"width": 4, "length": 2, "height": 1},
      "value": 18,
      "weight": 6,
      "is_fragile": false,
      "eta": 3
    },
    {
      "id": "B",
      "dimensions": {"width": 2, "length": 2, "height": 1},
      "value": 14,
      "weight": 4,
      "is_fragile": false,
      "eta": 0
    },
    {
      "id": "C",
      "dimensions": {"width": 2, "length": 2, "height": 1},
      "value": 13,
      "weight": 4,
      "is_fragile": false,
      "eta": 2
    },
    {
      "id": "D",
      "dimensions": {"width": 1, "length": 2, "height": 1},
      "value": 9,
      "weight": 2,
      "is_fragile": true,
      "eta": 0
    },
    {
      "id": "E",
      "dimensions": {"width": 1, "length": 2, "height": 1},
      "value": 8,
      "weight": 2,
      "is_fragile": false,
      "eta": 4
    },
    {
      "id": "F",
      "dimensions": {"width": 3, "length": 3, "height": 2},
      "value": 25,
      "weight": 11,
      "is_fragile": false,
      "eta": 1
    }
  ]
}
```

Listing 23: Kasus uji `single_truck_constrained.json`

```
{
  "trucks": [
    {
      "id": "T1",
      "dimensions": {"width": 8, "length": 6, "height": 4},
      "max_capacity": 30
    }
  ],
  "packages": [
    {"id": "P1", "dimensions": {"width": 4, "length": 3, "height": 2}, "value": 32, "weight": 10, "is_fragile": false, "eta": 3},
    {"id": "P2", "dimensions": {"width": 3, "length": 2, "height": 2}, "value": 26, "weight": 8, "is_fragile": true, "eta": 0},
    {"id": "P3", "dimensions": {"width": 5, "length": 2, "height": 1}, "value": 25, "weight": 7, "is_fragile": false, "eta": 2},
    {"id": "P4", "dimensions": {"width": 2, "length": 2, "height": 3}, "value": 24, "weight": 6, "is_fragile": false, "eta": 1},
    {"id": "P5", "dimensions": {"width": 3, "length": 3, "height": 1}, "value": 20, "weight": 5, "is_fragile": true, "eta": 4},
    {"id": "P6", "dimensions": {"width": 2, "length": 4, "height": 1}, "value": 18, "weight": 4, "is_fragile": false, "eta": 0},
    {"id": "P7", "dimensions": {"width": 2, "length": 2, "height": 2}, "value": 17, "weight": 5, "is_fragile": false, "eta": 2},
    {"id": "P8", "dimensions": {"width": 1, "length": 3, "height": 2}, "value": 13, "weight": 3, "is_fragile": true, "eta": 1},
    {"id": "P9", "dimensions": {"width": 4, "length": 1, "height": 1}, "value": 12, "weight": 3, "is_fragile": false, "eta": 5},
    {"id": "P10", "dimensions": {"width": 2, "length": 1, "height": 1}, "value": 9, "weight": 2, "is_fragile": false, "eta": 0},
    {"id": "P11", "dimensions": {"width": 1, "length": 1, "height": 3}, "value": 8, "weight": 2, "is_fragile": false, "eta": 3},
    {"id": "P12", "dimensions": {"width": 9, "length": 1, "height": 1}, "value": 40, "weight": 4, "is_fragile": false, "eta": 0}
  ]
}
```

Listing 24: Kasus uji `multi_truck_constrained.json`

```
{
  "trucks": [
    {
      "id": "T-small",
      "dimensions": {"width": 5, "length": 4, "height": 3},
      "max_capacity": 12
    },
    {
      "id": "T-medium",
      "dimensions": {"width": 7, "length": 5, "height": 4},
      "max_capacity": 20
    },
    {
      "id": "T-large",
      "dimensions": {"width": 9, "length": 6, "height": 5},
      "max_capacity": 28
    }
  ],
  "packages": [
    {"id": "M1", "dimensions": {"width": 8, "length": 5, "height": 2}, "value": 55, "weight": 16, "is_fragile": false, "eta": 4},
    {"id": "M2", "dimensions": {"width": 6, "length": 4, "height": 3}, "value": 42, "weight": 12, "is_fragile": false, "eta": 1},
    {"id": "M3", "dimensions": {"width": 5, "length": 4, "height": 2}, "value": 34, "weight": 10, "is_fragile": false, "eta": 3},
    {"id": "M4", "dimensions": {"width": 4, "length": 3, "height": 3}, "value": 30, "weight": 9, "is_fragile": false, "eta": 0},
    {"id": "M5", "dimensions": {"width": 7, "length": 2, "height": 2}, "value": 28, "weight": 8, "is_fragile": false, "eta": 2},
    {"id": "M6", "dimensions": {"width": 3, "length": 3, "height": 2}, "value": 24, "weight": 7, "is_fragile": true, "eta": 0},
    {"id": "M7", "dimensions": {"width": 2, "length": 5, "height": 2}, "value": 22, "weight": 6, "is_fragile": false, "eta": 5},
    {"id": "M8", "dimensions": {"width": 2, "length": 2, "height": 3}, "value": 20, "weight": 5, "is_fragile": true, "eta": 1},
    {"id": "M9", "dimensions": {"width": 4, "length": 2, "height": 1}, "value": 18, "weight": 4, "is_fragile": false, "eta": 2},
    {"id": "M10", "dimensions": {"width": 3, "length": 2, "height": 2}, "value": 17, "weight": 4, "is_fragile": false, "eta": 3},
    {"id": "M11", "dimensions": {"width": 2, "length": 2, "height": 2}, "value": 15, "weight": 3, "is_fragile": false, "eta": 0},
    {"id": "M12", "dimensions": {"width": 10, "length": 2, "height": 1}, "value": 60, "weight": 10, "is_fragile": false, "eta": 0}
  ]
}
```